DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA
IN
MODELLI E METODI PER LA SICUREZZA DELLE APPLICAZIONI

Phishing: Sviluppo di una estensione Chrome per il riconoscimento di siti malevoli mediante AI

RELATORE:
Prof. Donato Impedovo

LAUREANDO:
Nicolò Orsi

CORRELATORI:
Dott. Vincenzo Gattulli

ANNO ACCADEMICO 2024 - 2025

Abstract

Il presente lavoro di tesi illustra la progettazione, lo sviluppo e la valutazione di un sistema intelligente per il rilevamento in tempo reale di siti web di Phishing, integrato all'interno di un'estensione per browser denominata *BOOMERAngel*. Il nucleo di ricerca e di elaborazione dati del progetto è stato interamente sviluppato e documentato all'interno di un notebook esplorativo, il quale delinea una rigorosa pipeline di Data Science delineata in tre fasi principali: Estrazione delle Feature, Addestramento dei Modelli e Valutazione delle Performance.
Nella prima fase, è stato costruito un dataset bilanciato composto da 10.000 campioni (5.000 URL legittimi e 5.000 URL di phishing). Al fine di garantire un'analisi rapida e compatibile con i requisiti di latenza di un'estensione web, l'estrazione delle feature si è concentrata esclusivamente sulle proprietà lessicali e strutturali della barra degli indirizzi (URL), senza la necessità di ispezionare il contenuto della pagina o il traffico di rete.
Nella fase di modellizzazione, il dataset è stato impiegato per addestrare e confrontare molteplici architetture di Machine Learning e Deep Learning. Sono stati implementati e ottimizzati classificatori tradizionali, tra cui Random Forest, Support Vector Machine (SVM), Decision Tree, affiancati a metodi ensemble avanzati come XGBoost,LightGBM,Catboost e a reti neurali ricorrenti (LSTM) adatte all'analisi sequenziale dei caratteri dell'URL.
Infine, l'analisi comparativa dei risultati ha permesso di valutare i modelli sulla base di metriche standard quali Accuratezza e F1-Score. Il modello risultato più performante e robusto è stato successivamente serializzato e interfacciato con un server backend (Flask), fungendo così da motore inferenziale dell'estensione *BOOMERAngel*. Questo approccio dimostra come l'analisi lessicale degli URL, supportata da algoritmi di Machine Learning, rappresenti una valida e reattiva prima linea di difesa contro le frodi telematiche, proteggendo proattivamente la navigazione dell'utente finale.

Sommario

Capitolo 1 : Introduzione 6
1.1 Il Contesto: L'Evoluzione delle Minacce Online e l'Ingegneria Sociale 6
1.2 I Limiti delle Difese Tradizionali 7
1.3 L'Intelligenza Artificiale come Paradigma Innovativo 7
1.4 Algoritmi di Machine Learning per la Sicurezza 8
1.5 Obiettivo della Tesi: Un'Estensione Browser Intelligente 10

Capitolo 2 : Stato dell'arte 12

Capitolo 3 : Preparazione del Dataset 19
3.1 L'importanza della Qualità del Dato nel Machine Learning 19
3.2 Fonti dei Dati e Acquisizione 19
3.3 Composizione e Bilanciamento del Dataset 19
3.4 Pre-elaborazione e Pulizia dei Dati (Data Cleaning) 20
3.5 Identificazione delle Variabili per l'Analisi 20
3.6 Data Splitting e Preparazione all'Addestramento 21

Capitolo 4 – Metodologia e Modelli Predittivi 22
4.1 – Pre-Processing e Feature Engineering Strategico 22
4.2 – Modelli di Classificazione e Paradigmi di Apprendimento 26
4.3 – Dettagli implementativi 30

Capitolo 5 – Risultati 39
Accuratezza (Accuracy) 40
F1-Score 41
Precisione (Precision) 41
Richiamo (Recall) 42

Capitolo 6 – Estensione Chrome 43
6.1 – Panoramica Architetturale e Paradigma dell'Edge Computing 43
6.2 – L'Implementazione Predittiva: La Scelta di LightGBM 45
6.3 – Struttura Modulare e Componenti dell'Estensione 46

Capitolo 7 – Conclusioni 64

Ringraziamenti 66

Bibliografia 67

Capitolo 1 : Introduzione

1.1 Il Contesto: L'Evoluzione delle Minacce Online e l'Ingegneria Sociale

Nell'era sempre più interconnessa e digitalizzata, le applicazioni software e la navigazione web rappresentano l'essenza della nostra esperienza tecnologica quotidiana. Con la transizione verso il paradigma del *Cloud Computing* e l'affermazione dei servizi SaaS (Software as a Service), il browser web ha smesso di essere un semplice visualizzatore di documenti ipertestuali, trasformandosi in un vero e proprio sistema operativo universale. Attraverso di esso, gli utenti gestiscono quotidianamente operazioni bancarie, comunicazioni aziendali, cartelle cliniche e acquisti e-commerce. Tuttavia, con l'aumento esponenziale della dipendenza dai servizi online, emergono anche sfide significative e proporzionali legate alla sicurezza dell'informazione.

Gli attacchi informatici costituiscono una minaccia sempre più sofisticata, mettendo a repentaglio la privacy degli utenti e la fiducia nell'ambiente digitale. In passato, i criminali informatici si concentravano principalmente sullo sfruttamento di vulnerabilità tecniche (come i *buffer overflow* o le *SQL injection* dirette ai server); oggi, l'evoluzione costante delle difese perimetrali ha reso questi attacchi più complessi e costosi. Di conseguenza, il focus degli attaccanti si è spostato sull'anello intrinsecamente più debole della catena di sicurezza: l'essere umano.

Tra le minacce più pervasive, subdole e dannose spiccano gli attacchi di *phishing*. Questa forma di attacco informatico non forza i sistemi dall'esterno, ma si basa sull'ingegneria sociale: sfrutta la vulnerabilità umana, manipolando psicologicamente la vittima per indurla a cedere volontariamente dati sensibili, credenziali finanziarie e informazioni personali. L'aggressore crea un senso di urgenza, paura o curiosità (ad esempio, un finto blocco del conto corrente o una spedizione non consegnata), spingendo l'utente a compiere un'azione impulsiva. Come sottolineato nel testo di riferimento "The Art of Deception: Controlling the Human Element of Security" di Kevin D. Mitnick [23], la sicurezza non è solo un problema tecnologico, ma un problema umano; l'ingegneria sociale sfrutta la naturale propensione umana alla fiducia, rendendo inefficaci anche i firewall più avanzati se l'utente decide di aprire la porta all'attaccante.

Secondo i rapporti annuali dell'Anti-Phishing Working Group (APWG), il volume di questi attacchi continua a registrare livelli record, con impatti economici devastanti su scala globale. Un testo chiave per comprendere le sfaccettature di questo fenomeno è "Phishing: Cutting the Identity Theft Line" di Rachael Lininger e Russell Dean Vines [21], in cui viene fornita una panoramica approfondita sulle origini del phishing, analizzando le tecniche utilizzate dagli attaccanti e le strategie di difesa necessarie per proteggere identità digitali sempre più esposte. Inoltre, come evidenziato dall'articolo "A Survey of Phishing

Attacks: Their Types, Vectors, and Technical Approaches to Mitigate" di Shahrzad Rezaei et al. [22], i vettori di attacco si sono moltiplicati in modo allarmante, estendendosi dalle tradizionali email ai social network e alle piattaforme di messaggistica istantanea.

1.2 I Limiti delle Difese Tradizionali

Le minacce legate al furto di credenziali richiedono approcci di difesa altamente avanzati e reattivi. L'identificazione tempestiva e precisa di queste minacce è diventata cruciale per la preservazione dell'integrità dei dati. Tuttavia, le metodologie di sicurezza convenzionali spesso si scontrano con la rapida mutazione degli attacchi.

Storicamente, la difesa contro gli URL maligni si è basata sull'uso di *blacklist* (liste nere), database centralizzati mantenuti da organizzazioni di sicurezza che catalogano i siti fraudolenti noti. Sebbene indispensabili, i meccanismi di difesa basati su firme e liste statiche risultano oggi insufficienti contro attacchi sofisticati che si adattano dinamicamente. Le *blacklist* presentano infatti una latenza intrinseca, nota come "finestra di vulnerabilità": dal momento in cui un sito di phishing viene pubblicato online al momento in cui viene scoperto, analizzato, verificato e inserito nella lista nera, possono trascorrere ore o persino giorni. Durante questo lasso di tempo (*zero-day phishing*), gli utenti sono completamente indifesi.

Inoltre, gli attaccanti utilizzano tecniche di offuscamento avanzate per eludere i controlli, come l'uso di URL *shorteners* (servizi di abbreviazione), il *typosquatting* (la registrazione di domini con errori di battitura simili a quelli legittimi, es. "https://www.google.com/search?q=g00gle.com" anziché "https://www.google.com/search?q=google.com") o attacchi omoglifi, che sostituiscono caratteri latini con caratteri cirillici visivamente identici. Di conseguenza, emerge l'urgente necessità di adottare approcci più dinamici, che non si limitino a confrontare un URL con un elenco statico, ma che siano in grado di analizzarne la natura e prevederne la pericolosità in tempo reale.

1.3 L'Intelligenza Artificiale come Paradigma Innovativo

L'impiego di algoritmi di intelligenza artificiale e tecniche analitiche avanzate rappresenta il passo cruciale verso la costruzione di un sistema di difesa in grado di anticipare e contrastare le minacce emergenti. L'intelligenza artificiale (IA) è un campo dell'informatica che si occupa dello sviluppo di sistemi e tecnologie capaci di simulare alcune capacità cognitive umane. Come afferma Pedro Domingos, autore del libro "The Master Algorithm" [1], l'IA è la scienza di fare macchine intelligenti, specialmente programmi software intelligenti. È l'arte di creare macchine che eseguono attività che richiedono intelligenza quando fatte dall'uomo; queste attività includono il ragionamento, l'apprendimento, il riconoscimento di pattern, la comprensione del linguaggio naturale e il problem solving.

Diventata un argomento di grande rilevanza nella società moderna, l'IA sta influenzando profondamente molti aspetti della vita umana. Come sottolinea Max Tegmark nel suo libro "Life 3.0: essere umani nell'era dell'intelligenza artificiale" [2], l'IA è la più grande invenzione dell'umanità che amplificherà il nostro potenziale come nessuna tecnologia ha fatto fino ad ora. Come afferma Stephen Hawking nel suo libro "Brief Answer to the Big Questions" [5], l'IA potrebbe diventare il più grande evento nella storia della nostra civiltà; tuttavia, come ogni avanzamento tecnologico, presenta dei rischi, perciò è fondamentale investire nella ricerca di sistemi sicuri e affidabili.

Secondo il testo di riferimento accademico "Artificial Intelligence: A Modern Approach" di Stuart Russel e Peter Norvig [6], le applicazioni moderne di IA sono supportate da algoritmi di apprendimento automatico che consentono alle macchine di apprendere dai dati empirici e migliorare le loro prestazioni nel tempo, senza richiedere una programmazione esplicita basata su regole if-then. In questo contesto, il **Machine Learning (ML)** rappresenta la branca dell'IA più adatta a risolvere il problema del rilevamento del phishing. Questo processo di apprendimento è fondamentale per consentire ai sistemi di adattarsi a nuove informazioni e di prendere decisioni autonome. Una delle caratteristiche centrali del Machine Learning è la capacità di identificare pattern e relazioni nascoste all'interno di vaste moli di dati strutturati (Big Data).

Il processo di apprendimento può essere suddiviso in diverse tecniche:

1. **Apprendimento Supervisionato:** approccio in cui il modello viene addestrato utilizzando un insieme di dati di input e le rispettive etichette di output corrispondenti. L'obiettivo è insegnare al modello la funzione matematica che lega gli input agli output desiderati, in modo che possa generalizzare correttamente su nuovi dati non visti durante l'addestramento. Nel contesto di questa tesi, rappresenta l'approccio principale: al modello verranno forniti migliaia di URL etichettati come "phishing" o "legittimi" affinché impari a classificarli autonomamente.

2. **Apprendimento Non Supervisionato:** coinvolge l'addestramento del modello utilizzando solo dati di input, senza etichette di output. L'obiettivo principale è scoprire pattern, strutture di raggruppamento (*clustering*) o relazioni nascoste nei dati, utile ad esempio per individuare nuove famiglie di attacchi anomali non ancora categorizzati.

3. **Apprendimento Semi-Supervisionato:** combina elementi dei due approcci precedenti, addestrando il modello su un dataset misto. È estremamente utile in ambito cybersecurity, dove la raccolta di dati grezzi è semplice, ma l'etichettatura manuale accurata da parte di esperti richiede un grande dispendio di tempo e risorse.

1.4 Algoritmi di Machine Learning per la Sicurezza

L’adozione diffusa del Machine Learning è stata resa possibile da una serie di fattori tecnologici, tra cui l’aumento esponenziale della potenza di calcolo (grazie all'uso di GPU e processori specializzati) e la disponibilità di enormi dataset. Queste considerazioni sono supportate da autori illustri, tra cui Aurelien Geron nel suo libro “Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow” [7], che fornisce un'ampia panoramica delle metodologie di addestramento. Allo stesso modo, “Pattern Recognition and Machine Learning” di Christopher M. Bishop [8] approfondisce i principi statistici fondamentali che governano questi modelli.

Sono molteplici gli usi degli algoritmi di machine learning (dalle raccomandazioni personalizzate di piattaforme come Netflix all'ottimizzazione della logistica), ma nell'ambito della classificazione binaria (URL sicuro vs. URL malevolo) assumono particolare rilevanza i seguenti modelli:

- **Regressione Logistica:** È un metodo statistico utilizzato per modellare la relazione tra una variabile dipendente binaria e una o più variabili indipendenti. Secondo "Introduction to Statistical Learning" di Gareth James et al. [16], la regressione logistica è largamente apprezzata per la sua interpretabilità. Utilizzando la funzione sigmoide, trasforma una combinazione lineare di *features* in un valore di probabilità compreso tra 0 e 1. La sua efficienza computazionale la rende ideale come modello di base (*baseline*) per valutare l'effettiva utilità di modelli più complessi.

- **K-Nearest Neighbors (K-NN):** È uno degli algoritmi non parametrici più intuitivi, basato sul calcolo della distanza vettoriale (spesso euclidea). Si basa sull'assunzione che punti dati con caratteristiche simili si trovino vicini nello spazio multidimensionale. Secondo Bishop [8], la scelta del parametro K è cruciale per bilanciare *bias* e varianza. Tuttavia, come sottolineato in "Introduction to Machine Learning with Python" di Andreas C. Müller e Sarah Guido [13], una forte limitazione di K-NN è la sua inefficienza computazionale in fase di inferenza: dovendo ricalcolare le distanze per ogni nuovo campione, risulta poco scalabile e inadatto per un'esecuzione rapida all'interno di un browser.

- **Alberi Decisionali (Decision Tree):** Modelli che creano una struttura gerarchica ad albero, in cui ogni nodo interno rappresenta una divisione basata su una caratteristica dell'URL (es. "L'URL è più lungo di 75 caratteri?"). Secondo Müller e Guido [13], sono estremamente veloci sia nell'addestramento che nella predizione e non richiedono una complessa normalizzazione dei dati. Tuttavia, la loro natura li porta spesso a memorizzare troppo fedelmente i dati di addestramento, causando problemi di *overfitting* se non si applicano tecniche di *pruning* (potatura).

- **Random Forest:** È un robusto algoritmo *ensemble* (di aggregazione) che crea una "foresta" di molteplici alberi decisionali, ognuno addestrato su sottoinsiemi casuali di dati e caratteristiche. Durante la predizione, il risultato finale viene stabilito

tramite votazione maggioritaria tra tutti gli alberi. Come evidenziato da Bishop [8], il Random Forest compensa brillantemente i difetti dei singoli alberi, riducendo drasticamente l'overfitting e gestendo con eccellenza dati rumorosi e sbilanciati.

- **Support Vector Machine (SVM):** È un algoritmo potente che mira a trovare l'iperpiano che massimizza il margine di separazione tra le diverse classi di dati. Secondo Géron [7], una delle peculiarità delle SVM è l'uso del *kernel trick*, che consente di proiettare i dati in dimensioni superiori per separare in modo lineare dati che originariamente non lo erano. Offre un'eccezionale capacità di generalizzazione, fondamentale per identificare varianti inedite di phishing.

1.5 Obiettivo della Tesi: Un'Estensione Browser Intelligente

Nonostante l'enorme potenziale di questi algoritmi, la sfida ingegneristica principale risiede nella loro effettiva implementazione e distribuzione agli utenti finali. Attualmente, gran parte delle soluzioni di sicurezza basate sull'IA operano lato server, analizzando il traffico di rete in punti di snodo centralizzati. Questo approccio presenta tuttavia delle criticità: solleva preoccupazioni in materia di privacy (poiché la cronologia di navigazione dell'utente deve essere inviata a terzi) e introduce una latenza di rete che, per quanto minima, può ritardare il blocco della pagina fraudolenta.

Il presente lavoro di tesi si propone di affrontare queste sfide proponendo un cambio di paradigma: spostare l'intelligenza artificiale direttamente sul dispositivo dell'utente (*Edge Computing*). L'obiettivo fondamentale è la progettazione, l'addestramento e lo sviluppo di un sistema di classificazione integrato all'interno di un'**estensione per il browser Google Chrome**.

Tale sistema utilizzerà i modelli di machine learning precedentemente descritti per identificare e distinguere con precisione gli attacchi di phishing in tempo reale. Estrarre le caratteristiche (*features*) lessicali e strutturali direttamente dal browser del client, ed eseguire lì l'inferenza del modello, garantisce una risposta immediata e mirata, proteggendo l'utente prima ancora che possa interagire con gli elementi malevoli della pagina web.

Non solo si punta a identificare e analizzare le nuove minacce, ma anche a proporre una soluzione software concreta e usabile. Nel perseguire questo obiettivo, il presente elaborato si articolerà come segue: Il **Capitolo 2** presenterà un'attenta analisi dello Stato dell'Arte, confrontando i recenti studi accademici che impiegano il Machine Learning in combinazione con estensioni web. Il **Capitolo 3** esplorerà la metodologia di ricerca, descrivendo il processo di acquisizione del dataset, la pulizia dei dati e la complessa fase di *Feature Engineering* necessaria per parametrizzare gli URL. Il **Capitolo 5** illustrerà i risultati della fase sperimentale, con un confronto rigoroso delle metriche di accuratezza, precisione e richiamo dei vari algoritmi addestrati. Il **Capitolo 6** sarà dedicato all'architettura software dell'estensione Google Chrome, spiegandone il ciclo di vita, l'integrazione del modello predittivo esportato e la gestione delle interfacce utente. Infine,

il **Capitolo 7** trarrà le conclusioni dell'intero progetto, delineando i traguardi raggiunti e i possibili scenari di sviluppo futuro per la lotta contro le frodi digitali.

Capitolo 2 : Stato dell’arte

Nell’articolo “Real-time phishing detection using deep learning methods by extensions”[18] si propone un metodo per la rilevazione in tempo reale del phishing attraverso l'uso di estensioni per browser che impiegano modelli di deep learning, in particolare reti neurali convoluzionali (CNN). L'estensione protegge gli utenti verificando, avvisando e prevenendo connessioni non affidabili. I risultati ottenuti mostrano un'accuratezza del 98.4% nella rilevazione di URL maligni utilizzando una CNN.
Processo di Addestramento:
Il processo di addestramento del modello è suddiviso in più fasi:

1. Preelaborazione dei Dati:
    - Raccolta di un dataset di URL contenente 651,191 campioni, suddivisi in URL benigni, defacement, phishing e malware.
    - Estrazione delle caratteristiche degli URL come protocollo, sottodominio, nome di dominio, porta, percorso, query, parametri e frammento.
    - Tokenizzazione degli URL utilizzando un dizionario di 270 caratteri per convertire gli URL in vettori numerici.
2. Architettura del Modello:
    - CNN: La rete neurale convoluzionale è composta da livelli convoluzionali per l'estrazione delle caratteristiche, livelli di pooling per ridurre la dimensionalità e un livello completamente connesso per la classificazione finale.
    - CNN-LSTM: Una variante che include un livello LSTM per catturare le dipendenze temporali nei dati, utile per analizzare sequenze di caratteri negli URL.
3. Fasi di Valutazione:
    - Metriche di Valutazione: Accuratezza, precisione, richiamo e F1-score.
    - Configurazione Hardware: Gli esperimenti sono stati condotti utilizzando server con configurazioni hardware avanzate, tra cui CPU a 36 core, 64 GB di RAM e GPU NVIDIA® GeForce RTX™ 3060.

Estensione del Browser:
L'estensione software, sviluppata in JavaScript, è installata sui browser con kernel Chromium (es. Google Chrome e Microsoft Edge). Le sue principali funzioni includono:

- Invio dell'URL al Server: L'URL inserito dall'utente viene inviato al server per la classificazione.
- Classificazione in Tempo Reale: Il server utilizza il modello CNN per classificare l'URL come benigno o phishing.
- Notifica all'Utente: Se l'URL è maligno, l'estensione notifica immediatamente l'utente con un avviso.

Risultati Sperimentali:

- Accuratezza del Modello: La CNN ha raggiunto un'accuratezza del 98.4%, superando altri modelli come la regressione logistica (LR), gli alberi decisionali (DT), le foreste casuali (RF) e le macchine a vettori di supporto (SVM).
- Confronto tra Modelli: La CNN ha dimostrato un'elevata precisione e richiamo

nella rilevazione degli URL di phishing grazie alla sua architettura e all'uso di un set di caratteri esteso.

L'implementazione di un'estensione per browser basata su deep learning, in particolare CNN, offre un metodo efficace per la rilevazione in tempo reale degli URL maligni. Questo approccio migliora significativamente la sicurezza degli utenti durante la navigazione web. Il modello CNN ha dimostrato superiorità rispetto ad altri modelli di machine learning testati, evidenziando la sua efficacia nell'identificazione delle minacce informatiche.

L'articolo "Classification of Malicious Websites Using Machine Learning Based on URL Characteristics"[19] presenta un approccio basato su machine learning per la classificazione di siti web maligni utilizzando le caratteristiche degli URL. Utilizzando un dataset di URL aggiornato, l'algoritmo Random Forest ha raggiunto un'accuratezza del 95.68% nel rilevare siti web maligni. Il software sviluppato per la rilevazione e l'avviso di siti web maligni è implementato sia come applicazione web che come estensione del browser.

Il processo di addestramento del modello di rilevamento dei siti web maligni è suddiviso in due fasi principali:

1. Fase di Addestramento:
    - Dataset: Il dataset di addestramento include 213,345 URL suddivisi in cinque categorie: siti benigni, defacement, phishing, malware e spam.
    - Estrazione delle Caratteristiche: Vengono estratte 20 caratteristiche principali dagli URL, come la presenza di un indirizzo IP, la lunghezza dell'URL, l'uso di servizi di accorciamento, la presenza del protocollo HTTPS, il numero di cifre e l'anomalia dell'URL.
    - Algoritmi di Machine Learning: Gli algoritmi utilizzati includono Decision Tree, Random Forest, K-Nearest Neighbors (K-NN) e Adaptive Boost (AdaBoost).
2. Fase di Rilevamento:
    - Estrazione delle Caratteristiche: Gli URL vengono monitorati ed estratti utilizzando il classificatore addestrato.
    - Classificazione: Gli URL vengono classificati come benigni, defacement, phishing, malware o spam.

Algoritmi di Classificazione:

- Decision Tree: Utilizza un diagramma ad albero per prendere decisioni basate su eventi casuali, spese di risorse e benefici.
- Random Forest: Genera alberi decisionali su campioni di dati selezionati casualmente, predice per ogni albero e seleziona la migliore decisione tramite votazione.
- K-Nearest Neighbors (K-NN): Algoritmo di apprendimento supervisionato che classifica gli oggetti basandosi sulla vicinanza ai vicini più prossimi nel dataset.
- Adaptive Boost (AdaBoost): Algoritmo potente che accelera la creazione di un forte classificatore selezionando buone caratteristiche da una famiglia di deboli classificatori e combinandoli linearmente.

Dataset:

Il dataset utilizzato comprende 213,345 URL, suddivisi in cinque categorie: Benign (0),

Defacement (1), Phishing (2), Malware (3) e Spam (4). Gli URL maligni sono stati raccolti da fonti come OpenPhish, Phishtank, Zone-H e WEBSPAM-UK2007. Il processo di pulizia dei dati ha rimosso gli URL duplicati e quelli che non includevano tutte le caratteristiche selezionate.
Risultati e Discussione:

- Valutazione delle Performance: Gli algoritmi di machine learning sono stati valutati utilizzando 10-fold cross-validation. Le metriche di performance utilizzate includono Precision, Recall, accuratezza e F1-Score.
- Random Forest: L'algoritmo Random Forest ha ottenuto i migliori risultati con un'accuratezza del 94.8388% e un F1-Score del 95%. La matrice di confusione normalizzata ha mostrato un'alta precisione di riconoscimento dei dati etichettati, con un tasso di falsi riconoscimenti relativamente basso, il più alto essendo l'1.59%.

Implementazione Software:

- Applicazione Web: Implementata utilizzando JavaScript e Python, l'interfaccia permette di rilevare e avvisare l'utente su siti web maligni.
- Estensione del Browser: Sviluppata utilizzando HTML, JavaScript e CSS, l'estensione utilizza il modello Random Forest implementato in Python per rilevare i siti web maligni in tempo reale.

L'articolo dimostra che l'uso di metodi di machine learning per la classificazione di siti web maligni basati sulle caratteristiche degli URL è efficace. Il metodo Random Forest ha mostrato le migliori performance. Il software sviluppato, sia come applicazione web che come estensione del browser, offre una soluzione pratica per rilevare e avvisare gli utenti di siti web maligni.

Nell'articolo "Development of a malicious website detection system using web browser extension"[24] si discute lo sviluppo di un sistema per la rilevazione di siti web dannosi attraverso un'estensione del browser web. Il sistema proposto combina blacklist e tecniche di machine learning tramite estrazione di caratteristiche per rendere il software ancora più preciso e robusto nel riconoscimento di URL malevoli.
Il sistema è composto da tre livelli principali:

1. User Layer: Interfaccia che permette agli utenti di interagire con il browser e agli amministratori di aggiornare il database delle blacklist.
2. Web Browser Extension Layer: Nucleo del sistema che esegue la previsione della pericolosità del sito web utilizzando due tecniche:
    - Blacklist Feature: Controlla se l'URL visitato è presente nel database delle blacklist.
    - Feature Extractor: Estrae caratteristiche dall'HTML, JavaScript e contenuti lessicali del sito web per determinare se è dannoso.
3. Database Layer: Contiene informazioni di login degli amministratori e la lista delle blacklist.

Metodologie e Implementazione
Il sistema utilizza JavaScript per l'estensione del browser e MySQL per il database. La rilevazione avviene in due fasi:

- Controllo della Blacklist: Se l'URL è presente nella blacklist, viene segnalato come dannoso.

- Estrazione delle Caratteristiche: Se l'URL non è presente nella blacklist, il sistema analizza le caratteristiche del sito (HTML, JavaScript, contenuti lessicali) per determinarne la pericolosità.

L'articolo fornisce una soluzione pratica ed efficace per la rilevazione dei siti web dannosi. La combinazione di blacklist e tecniche di estrazione di caratteristiche garantisce una protezione robusta e in tempo reale contro le minacce web. Questo approccio è particolarmente utile in un contesto in cui le minacce evolvono rapidamente e i metodi tradizionali non sono sufficienti. La facile implementazione dell'estensione del browser e la sua capacità di aggiornarsi automaticamente tramite il database delle blacklist rendono questa soluzione praticabile e scalabile per un'ampia gamma di utenti e contesti di utilizzo.

L'articolo "Phishing Detection and Prevention using Chrome Extension"[25] tratta lo sviluppo di un'estensione Chrome per la rilevazione e la prevenzione del phishing. Durante la pandemia di COVID-19, gli attacchi di phishing sono aumentati significativamente, evidenziando la necessità di soluzioni tecniche più efficaci. L'approccio proposto utilizza algoritmi di machine learning, in particolare il Support Vector Machine (SVM), per migliorare l'accuratezza e l'adattabilità nel rilevare nuove minacce di phishing.
L'estrazione delle caratteristiche degli URL è fondamentale per determinare se un URL è dannoso. Il dataset utilizzato contiene 30 caratteristiche, ridotte a 16 per l'implementazione nel browser. Queste caratteristiche includono:

- isIPInURL: Identifica la presenza di un indirizzo IP nell'URL.
- isLongURL: Controlla se l'URL è più lungo di 75 caratteri.
- isTinyURL: Controlla se l'URL è più corto di 20 caratteri.
- isAlphaNumericURL: Cerca caratteri alfanumerici nell'URL.
- isRedirectingURL: Verifica la presenza di più "//" nell'URL.
- isHypenURL: Cerca "-" adiacenti al nome di dominio.
- isMultiDomainURL: Limita il nome di dominio al livello superiore e al codice del paese.
- isFaviconDomainUnidentical: Controlla se i link sulla pagina web provengono da un altro dominio.
- isIllegalHttpsURL: Identifica la presenza multipla di 'https' in una stringa URL.
- isImgFromDifferentDomain: Verifica se le immagini sono caricate da altri domini.
- isAnchorFromDifferentDomain: Controlla se i link sulla pagina web provengono da un altro dominio.
- isScLnkFromDifferentDomain: Verifica se gli script sulla pagina web provengono da un altro dominio.
- isFormActionInvalid: Cerca sottomissioni di form invalidi o vuoti.
- isMailToAvailable: Cerca tag anchor che incorporano mailto.
- isStatusBarTampered: Controlla se "onmouseover" controlla la visualizzazione della barra di stato.
- isIframePresent: Identifica le pagine che visualizzano iframes nel DOM.

Le caratteristiche sono estratte e codificate in valori {-1, 0, 1} (Legittimo, Sospetto, Phishing) in tempo reale durante il caricamento della pagina.
L'algoritmo di rilevamento e prevenzione del phishing utilizza tre modelli di machine learning supervisionati:

1. Random Forests
2. Artificial Neural Networks (ANN)
3. Support Vector Machines (SVM)

I modelli sono addestrati utilizzando Python con librerie come Scikit-learn, NumPy e Pandas. SVM è stato selezionato come classificatore appreso per la sua elevata accuratezza e specificità. L'algoritmo embedded nel browser predice la validità dei siti visitati e avverte l'utente in caso di URL dannosi.

Il dataset utilizzato include 11,055 URL, con 6157 casi di phishing e 4898 casi legittimi. I dati di phishing sono stati ottenuti da PhishTank e dal repository di UCI Machine Learning. L'algoritmo è stato testato in un ambiente di laboratorio controllato utilizzando un'estensione Chrome. Le metriche di performance valutate includono accuratezza, sensibilità e specificità. I risultati mostrano che SVM supera gli altri classificatori con un'accuratezza del 90.05% e una specificità del 93%.

Performance Matrix dei Classificatori

- Accuratezza: SVM (90.05%), Random Forest (89.63%), ANN (89.03%)
- Sensibilità: Random Forest (89%), SVM (86%), ANN (86%)
- Specificità: SVM (93%), ANN (92%), Random Forest (90%)

SVM ha mostrato il tasso più basso di falsi positivi tra i tre classificatori, rendendolo il più efficace per classificare URL di phishing.

L'estensione Chrome è progettata con un'interfaccia utente semplice utilizzando HTML e CSS. Permette agli utenti di scaricare e utilizzare l'estensione per rilevare phishing. In caso di rilevamento, l'estensione blocca il sito e avvisa l'utente, fornendo opzioni per ulteriori dettagli o per ignorare l'avviso.

I risultati dimostrano che l'algoritmo SVM implementato nell'estensione Chrome è efficace nel rilevare URL di phishing con alta accuratezza e bassa incidenza di falsi positivi. L'approccio basato su machine learning migliora significativamente rispetto ai metodi tradizionali, adattandosi rapidamente alle nuove minacce.

Nell'articolo "PhishShield: ML Based-Powered Phishing [Client-Side Protection Against Web Spoofing Attacks]"[26] si discute la progettazione e lo sviluppo di PhishShield, un'estensione per Google Chrome progettata per rilevare e prevenire attacchi di phishing basati su spoofing web. La motivazione alla base di questo lavoro è l'aumento degli attacchi di phishing, soprattutto durante la pandemia di COVID-19, che hanno messo in pericolo la sicurezza e la privacy degli utenti. PhishShield utilizza tecniche di machine learning, in particolare il classificatore Random Forest, per valutare la legittimità degli URL e classificare le pagine di login come sospette o affidabili.

L'architettura di PhishShield è composta da tre moduli principali:

1. Backend: Ospita il classificatore e gestisce il dataset.
2. Frontend: Contiene il codice eseguibile scritto in JavaScript, CSS e HTML.
3. Modulo Statico: Include file essenziali come testdata.json e classifier.json.

Il cuore dell'estensione è il file manifest (manifest.json) che definisce le informazioni cruciali dell'estensione come nome, versione, descrizione e permessi richiesti.

Le caratteristiche degli URL sono cruciali per determinare se un sito è di phishing. Il dataset utilizzato comprende informazioni da quattro fonti principali:

- Mohammad et al.: Caratteristiche efficaci per rilevare attacchi di phishing.

- Jalalian et al.: Una collezione di 90 siti web compromessi.
- PhishTank: Un set di 310 URL presenti nella blacklist.
- Moz.com/top500: Un set di 310 URL legittimi.

Le caratteristiche selezionate sono divise in quattro gruppi:

1. Caratteristiche basate sulla barra degli indirizzi
2. Caratteristiche basate su anomalie
3. Caratteristiche basate su HTML e JavaScript
4. Caratteristiche basate sul dominio

PhishShield utilizza il classificatore Random Forest, scelto per la sua robustezza ed efficienza nel rilevamento degli attacchi di phishing. Il processo di classificazione avviene direttamente nel browser del client, migliorando la privacy e riducendo la latenza.

Il processo di sviluppo di PhishShield comprende:

1. Selezione del Modello: Utilizzo di tecniche di data mining per differenziare tra pagine web legittime e di phishing.
2. Preparazione dei Dati: Raccolta e preparazione di dataset rilevanti per l'estrazione delle caratteristiche.
3. Estrazione delle Caratteristiche: Analisi e selezione delle caratteristiche migliori per il modello.
4. Addestramento del Modello: Addestramento del classificatore Random Forest utilizzando tecniche di machine learning supervisionato.

L'efficacia di PhishShield è stata valutata utilizzando un dataset di 800 URL, divisi equamente tra URL di phishing e legittimi. I risultati mostrano:

- Accuratezza: 98.5%
- Precisione: 98.5%
- Recall: 98.5%

PhishShield ha mostrato un tempo di risposta medio di 62.5 millisecondi durante i test su 40 URL di phishing. Questa latenza è significativamente inferiore rispetto agli strumenti stateful come SpoofCatch, che ha mostrato una latenza media di 512 millisecondi.

Il sistema proposto dimostra un'elevata efficacia nella rilevazione degli attacchi di phishing, con un'accuratezza del 98.5% e tempi di risposta rapidi. La classificazione eseguita direttamente nel browser del client offre vantaggi in termini di privacy e velocità.

L'articolo "BrowSEcure"[27] discute lo sviluppo di BrowSEcure, un'estensione per Chrome progettata per prevenire il furto di informazioni sensibili come credenziali di accesso, dettagli di carte di credito e dati personali da parte di siti web di phishing. BrowSEcure utilizza avvisi di sicurezza per segnalare agli utenti quando stanno navigando su URL potenzialmente dannosi, prevenendo così gli attacchi di phishing.

BrowSEcure utilizza algoritmi di machine learning per migliorare l'accuratezza e l'efficienza del rilevamento dei domini di phishing. Questi algoritmi analizzano varie caratteristiche degli URL, come quelle lessicali, relative al dominio e all'host, per identificare i modelli che distinguono i domini legittimi da quelli di phishing.

L'algoritmo di machine learning utilizzato per rilevare i domini di phishing è il Random Forest, che combina la potenza di più alberi decisionali per fare previsioni accurate. L'algoritmo viene addestrato su un ampio dataset etichettato contenente sia URL legittimi che di phishing. Durante l'addestramento, ogni albero decisionale apprende diversi aspetti

dei dati, migliorando la capacità di generalizzazione e riducendo il rischio di overfitting.

- Raccolta dei Dati: Include la raccolta di domini legittimi e di phishing, con informazioni rilevanti come nomi di dominio, indirizzi IP, certificati SSL e contenuti HTML.
- Preprocessing dei Dati: Pulizia e preparazione dei dati per l'analisi, gestendo valori mancanti e rimuovendo duplicati.
- Estrazione delle Caratteristiche: Analisi delle caratteristiche dei nomi di dominio, reputazione degli indirizzi IP, anomalie nei certificati SSL e struttura del contenuto delle pagine web.

Oltre al Random Forest, sono stati utilizzati e confrontati diversi algoritmi di machine learning per la rilevazione dei phishing:

- Decision Tree Classifier: 95% di accuratezza
- Support Vector Machine (SVM): 54% di accuratezza
- K-Nearest Neighbors: 68% di accuratezza
- Logistic Regression: 91% di accuratezza
- Naive Bayes: 90% di accuratezza
- AdaBoostClassifier: 93% di accuratezza
- Hybrid Ensembler: 94% di accuratezza

Il Random Forest ha dimostrato di essere l'algoritmo più efficace, con un'accuratezza del 97%.

L'estensione BrowSEcure per il browser Chrome integra vari algoritmi di machine learning per analizzare gli URL e classificarli come legittimi o sospetti. Utilizza anche database di phishing esistenti per confrontare gli URL in tempo reale. Quando viene rilevato un URL sospetto, l'estensione visualizza indicatori visivi e avvisi per aiutare gli utenti a prendere decisioni informate.

L'algoritmo di BrowSEcure è stato testato su un dataset di 7900 siti di phishing e 5800 siti legittimi. I risultati mostrano che l'algoritmo Random Forest ha ottenuto il miglior rendimento in termini di accuratezza.

L'estensione è progettata per operare in tempo reale, analizzando e classificando gli URL durante la navigazione degli utenti. La latenza del sistema è stata ottimizzata per garantire una rapida risposta agli utenti, fornendo avvisi tempestivi senza rallentare l'esperienza di navigazione.

BrowSEcure si è dimostrato efficace nel rilevare e prevenire attacchi di phishing utilizzando tecniche di machine learning. L'integrazione dell'estensione nel browser fornisce una difesa proattiva contro le minacce di phishing, migliorando la sicurezza online degli utenti. Il sistema continuerà a evolversi, incorporando feedback degli utenti e aggiornamenti regolari per affrontare nuove tecniche di phishing.

Per migliorare ulteriormente BrowSEcure, si prevede di:

- Ampliare il set di caratteristiche utilizzate per il rilevamento.
- Implementare altri classificatori discriminativi come SVM.
- Addestrare il modello su dataset più grandi.
- Integrare meccanismi di feedback degli utenti per migliorare continuamente le capacità di rilevamento.

Questi miglioramenti mirano a rafforzare le capacità di BrowSEcure e offrire agli utenti una protezione robusta contro le minacce di phishing in continua evoluzione

Capitolo 3 : Preparazione del Dataset

3.1 L'importanza della Qualità del Dato nel Machine Learning

La fase di raccolta e preparazione dei dati costituisce il pilastro fondamentale su cui poggia l'intera architettura di apprendimento automatico. Come evidenziato nella letteratura scientifica e nei testi fondamentali della disciplina, la capacità di generalizzazione di un modello — ovvero la sua abilità di fornire predizioni accurate su dati mai visti in precedenza — è direttamente proporzionale alla qualità, alla pulizia e alla rappresentatività del dataset di addestramento. Nel contesto specifico della cybersecurity e del rilevamento del phishing, la sfida risiede nel costruire una base di conoscenza che catturi non solo la "normalità" della navigazione web, ma anche le sottili anomalie strutturali impiegate dagli attaccanti

3.2 Fonti dei Dati e Acquisizione

Per lo sviluppo di questo sistema di classificazione, si è reso necessario assemblare un dataset eterogeneo che includesse un numero equo di campioni per le due classi di interesse: URL legittimi e URL di phishing. I dati grezzi sono stati acquisiti attraverso file CSV specializzati, caricati e manipolati tramite la libreria Python *Pandas*.
Le sorgenti utilizzate sono state le seguenti:
Dataset di URL Legittimi (legit_urls.csv): Questa collezione comprende indirizzi web sicuri e verificati, rappresentativi della struttura canonica dei domini più affidabili presenti in rete.
Dataset di URL di Phishing (phishing_urls.csv): Questa risorsa raccoglie URL malevoli segnalati e convalidati da repository di intelligence sulle minacce, contenenti pattern tipici di spoofing e offuscamento.

3.3 Composizione e Bilanciamento del Dataset

Al fine di garantire un addestramento imparziale ed evitare che i modelli sviluppassero un *bias* verso la classe maggioritaria, è stato applicato un processo di campionamento casuale. Come definito nel notebook di ricerca:
Sono stati estratti **5.000 campioni** per la classe degli URL legittimi.
Sono stati estratti **5.000 campioni** per la classe degli URL di phishing.
Il dataset finale risulta quindi composto da un totale di **10.000 record**.

Questo perfetto bilanciamento (50% per ogni classe) permette di valutare correttamente metriche fondamentali come la *Precision* e il *Recall*, assicurando che il classificatore impari a distinguere le caratteristiche distintive di entrambe le tipologie di URL con la medesima efficacia.

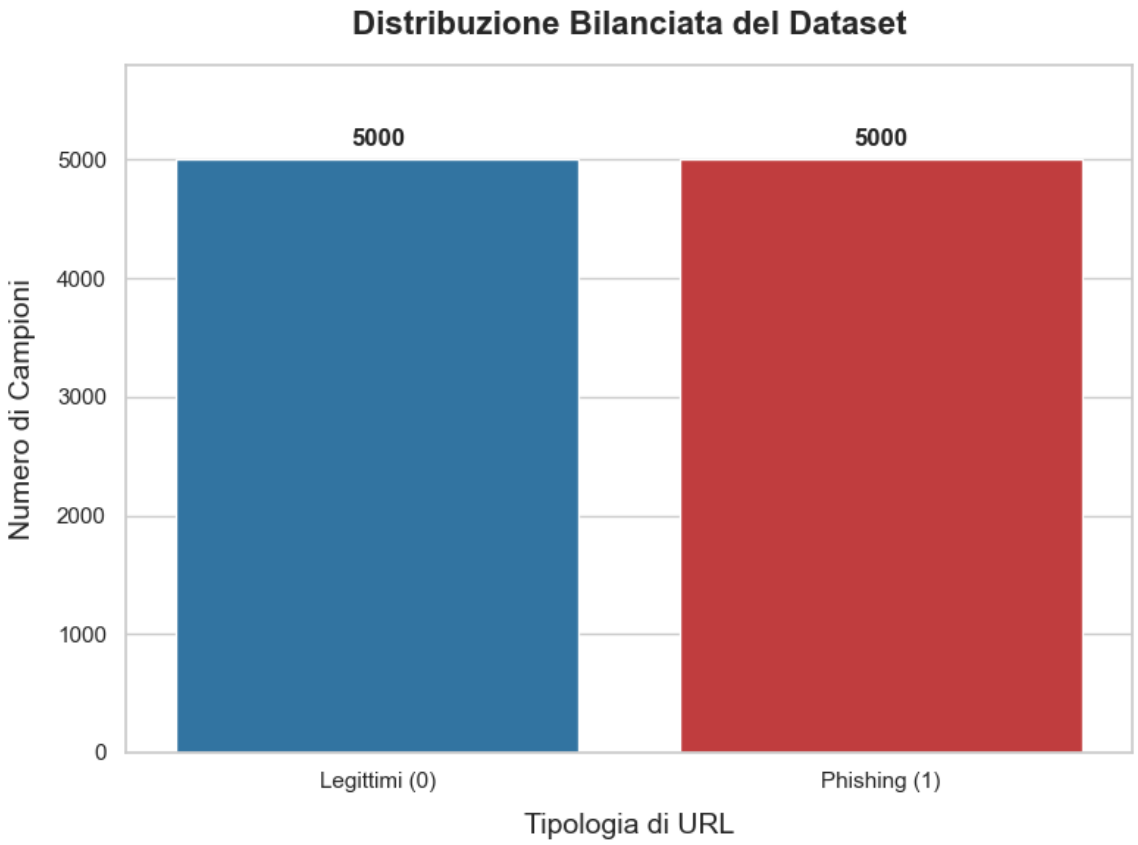


Figura 1 Distribuzione del Dataset

3.4 Pre-elaborazione e Pulizia dei Dati (Data Cleaning)

Prima di procedere all'estrazione delle caratteristiche, i dati hanno subito un rigoroso processo di pre-elaborazione per uniformarne il formato e garantirne l'integrità. Le operazioni principali eseguite includono:

- **Uniformazione delle Colonne:** I campioni sono stati raggruppati in un unico set di dati con una colonna standard denominata 'URL'.
- **Etichettatura (Labeling):** Ad ogni URL è stata associata l'etichetta target corrispondente. In conformità con gli standard di classificazione binaria, è stato assegnato il valore **0** agli URL legittimi e il valore **1** agli URL di phishing.
- **Reset dell'Indice:** Per mantenere la coerenza strutturale dei DataFrame dopo il campionamento, gli indici sono stati resettati eliminando i vecchi riferimenti.
- **Shuffling (Mescolamento Casuale):** Una volta uniti i due set, i dati sono stati rimescolati casualmente utilizzando la funzione sample(frac=1). Questa operazione è di vitale importanza per impedire che gli algoritmi di addestramento imparino pattern basati sull'ordine sequenziale dei campioni originali.

3.5 Identificazione delle Variabili per l'Analisi

Il dataset così preparato funge da input primario per il processo di *Feature Extraction*,

dove ogni stringa URL viene scomposta in 12 caratteristiche numeriche e booleane, accuratamente selezionate per garantire un'inferenza istantanea *client-side* all'interno dell'estensione browser. Tra queste, assumono particolare rilevanza l'Entropia di Shannon (cruciale per rilevare stringhe alfanumeriche casuali tipiche dei domini generati artificialmente), la presenza di parole chiave sensibili legate a tentativi di inganno (es. *'login'*, *'secure'*), l'uso di caratteri speciali di offuscamento (come il simbolo '@' o i trattini nel dominio) e il ricorso a servizi di abbreviazione (*TinyURL*). La corretta strutturazione e normalizzazione del dataset qui descritta ha permesso di raggiungere un duplice traguardo: da un lato ha garantito l'esecuzione fluida e offline di algoritmi classici ad alta efficienza (come LightGBM e Random Forest), e dall'altro ha posto le basi per l'addestramento sequenziale avanzato, permettendo di ottenere un'accuratezza superiore al 97% tramite l'algoritmo di Deep Learning LSTM.

3.6 Data Splitting e Preparazione all'Addestramento

Una volta completata la fase di pre-elaborazione e di *Feature Engineering*, è fondamentale preparare i dati per la fase di apprendimento. Nel campo del Machine Learning, valutare le prestazioni di un algoritmo sugli stessi dati utilizzati per il suo addestramento costituisce un errore metodologico grave, in quanto non permetterebbe di misurare la reale capacità di generalizzazione del modello (fenomeno dell'*overfitting*). Per tale motivo, il dataset globale deve essere partizionato in sottoinsiemi indipendenti.
La prima operazione eseguita è stata l'isolamento della variabile dipendente dalle variabili indipendenti. Dal DataFrame contenente i 10.000 campioni, è stata estratta la colonna Label per costituire il vettore target y (di dimensione (10000,)), mentre le restanti 12 caratteristiche ingegnerizzate hanno formato la matrice delle *features* X (di dimensione (10000, 12)).
La suddivisione del dataset è stata gestita diversamente in base alla tipologia di architettura addestrata, differenziando il Machine Learning classico dal Deep Learning.

3.6.1 Splitting per i Modelli di Machine Learning Classico

Per l'addestramento dei classificatori statistici (LightGBM, XGBoost, Random Forest, CatBoost, Decision Tree e SVM), la matrice X (contenente le features) e il vettore y (contenente le etichette) sono stati partizionati utilizzando la funzione train_test_split della libreria Scikit-Learn.
È stata adottata la rigorosa proporzione empirica dell'80/20, formalizzata impostando il parametro test_size = 0.2. Questa scelta garantisce un bacino di conoscenza sufficientemente ampio per far convergere gli algoritmi, riservando al contempo una mole di dati statisticamente rilevante per la validazione. La ripartizione ha generato i seguenti sottoinsiemi:
Training Set (X_train, y_train): Composto da 8.000 campioni (di dimensione (8000, 12)), impiegato esclusivamente per istruire i modelli e ottimizzarne gli iperparametri basandosi

sulle 12 caratteristiche lessicali estratte.
Test Set (X_test, y_test): Composto dai restanti 2.000 campioni (di dimensione (2000, 12)), tenuto rigorosamente "nascosto" durante la fase di apprendimento e utilizzato solo al termine per misurare le reali metriche di accuratezza, precisione e richiamo.
Inoltre, per garantire la totale riproducibilità degli esperimenti in sessioni di esecuzione future, è stato fissato il parametro random_state = 42. Questo accorgimento cristallizza il processo di partizionamento pseudo-casuale, assicurando che i campioni assegnati al set di addestramento e a quello di test rimangano invariati ad ogni esecuzione del codice.

3.6.2 Splitting e Set di Validazione per l'Architettura LSTM

Il partizionamento dei dati per la rete neurale LSTM ha richiesto un approccio più stratificato, dovuto alla natura profonda dell'algoritmo e all'utilizzo delle sequenze di token testuali (vettorializzate a una lunghezza fissa di 100 caratteri).
Anche in questo caso, le matrici contenenti le sequenze di URL convertite in interi (X) e le relative etichette (y) sono state inizialmente separate seguendo la medesima proporzione 80/20 (con random_state = 42) nei tensori X_train_lstm (di dimensione (8000, 100)) e X_test_lstm (di dimensione (2000, 100)).
Tuttavia, le reti neurali artificiali sono intrinsecamente prone all'overfitting (sovradattamento) se addestrate per troppe epoche sui medesimi dati. Per prevenire questo fenomeno, è stato introdotto un terzo sottoinsieme virtuale: il Validation Set.
Durante la compilazione della funzione model.fit(), è stato applicato il parametro validation_split = 0.2 al set di addestramento.
Ciò significa che il 20% degli 8.000 campioni di training (ovvero 1.600 URL) è stato escluso dall'aggiornamento dei pesi sinaptici e utilizzato al termine di ogni singola epoca (per un totale di 10 epoche) esclusivamente per calcolare la metrica di errore (val_loss).
Questo spacco in tempo reale ha permesso di implementare la tecnica dell'Early Stopping (con parametro patience = 3), la quale interrompe l'addestramento non appena la rete smette di migliorare sui dati di validazione, ripristinando i pesi ottimali raggiunti e garantendo la massima capacità di generalizzazione sull'invisibile Test Set.

Capitolo 4 – Metodologia e Modelli Predittivi

4.1 – Pre-Processing e Feature Engineering Strategico

La fase di Pre-Processing rappresenta uno degli snodi più critici nell'architettura di un sistema di rilevamento delle minacce. Nel contesto specifico di un'estensione per browser, il paradigma operativo si sposta dal tradizionale *Server-Side computing* all'*Edge Computing*. Ciò impone che l'estrazione delle caratteristiche avvenga in tempo reale, direttamente sul dispositivo dell'utente, senza dipendere dal rendering della pagina web o da latenti chiamate a database remoti. Pertanto, le operazioni di pre-elaborazione si concentrano esclusivamente sull'analisi statica, lessicale e matematica della stringa URL.

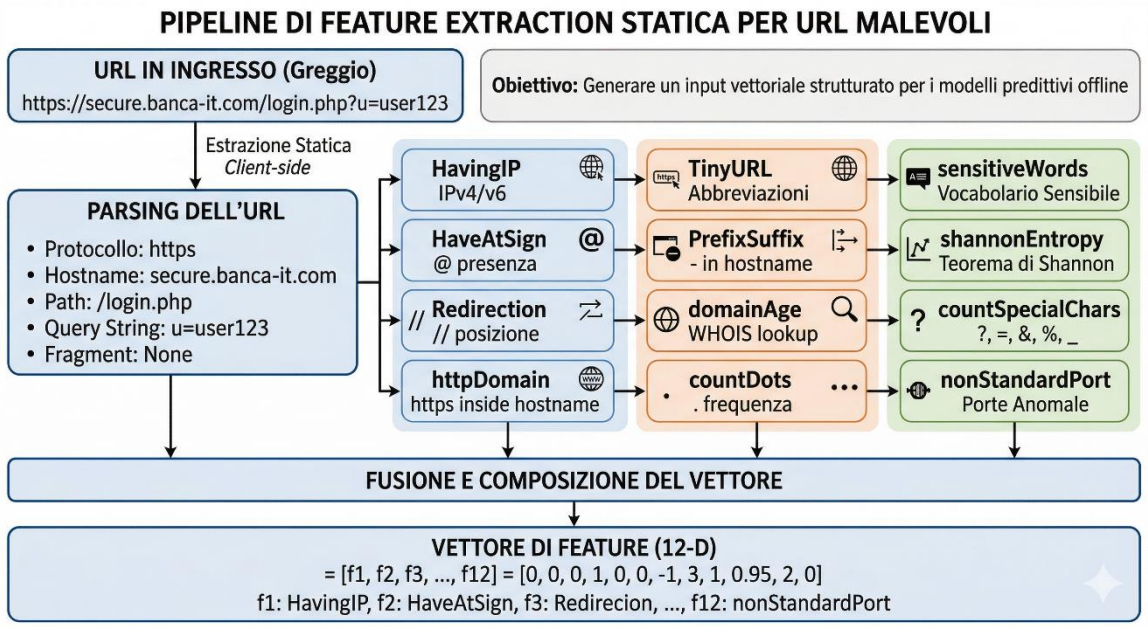


Figura 2 Diagramma di flusso della pipeline di Feature Extraction statica implementata in ambiente client-side.

Le operazioni eseguite in questa fase mirano a decostruire l'URL per rivelarne la natura, e includono:

1. **Normalizzazione e Pulizia:** Gli URL vengono intercettati e normalizzati per ridurre la complessità. Ad esempio, qualora la stringa sia sprovvista di protocollo esplicito, il sistema applica una correzione preventiva forzando l'inserimento del prefisso standard, assicurando che gli strumenti di *parsing* (come la manipolazione del DOM in JavaScript o urllib.parse in Python) possano identificare correttamente l'host.
2. **Parsing Strutturale:** L'URL viene suddiviso nei suoi token fondamentali: protocollo, dominio (netloc), percorso (path), parametri di query e frammenti.
3. **Analisi Semantica ed Entropica:** Vengono applicate euristiche avanzate per calcolare la casualità della stringa (misurata tramite il teorema di Shannon) e funzioni di corrispondenza per individuare la presenza di vocabolari tipici degli

attacchi di ingegneria sociale.

Questa rigorosa destrutturazione si concretizza nell'individuazione di **12 feature specifiche**, che costituiscono lo spazio vettoriale su cui i modelli statistici andranno ad operare:

• **havingIP:**	Negli URL legittimi, il nome a dominio maschera l'indirizzo IP del server grazie al protocollo DNS. I criminali utilizzano spesso indirizzi IP diretti (es. http://192.168.x.x/login) per eludere le blacklist basate sui nomi a dominio. Questa feature assume valore 1 se viene rilevato un formato IPv4 o IPv6.
• **haveAtSign:**	I browser web sono progettati per ignorare tutta la porzione di URL che precede il simbolo @. Gli attaccanti sfruttano questa vulnerabilità per creare stringhe ingannevoli (es. http://www.banca-sicura.it@dominio-malevolo.com), dove la vittima legge il nome legittimo, ma viene reindirizzata a quello fraudolento.
• **redirection:**	Il simbolo del doppio slash // dovrebbe comparire solo dopo il protocollo (in sesta o settima posizione). Se rilevato altrove, indica quasi certamente un tentativo di *Open Redirect* o un URL mascherato all'interno di parametri di query.
• **httpDomain:**	Una tattica di offuscamento semantico prevede l'inserimento della stringa "https" direttamente nel nome del dominio (es. http://https-secure-paypal.com), per indurre un falso senso di sicurezza nell'utente meno attento.
• **tinyURL:**	Gli URL shortener (come bit.ly o t.co) sono servizi legittimi ampiamente abusati dai phisher per nascondere il vero dominio di destinazione. Una Regex (Espressione Regolare) scansiona l'URL per rilevare l'utilizzo di oltre 30 noti servizi di abbreviazione.
• **prefixSuffix:**	La registrazione di un dominio legittimo è

	preclusa ai criminali. Per aggirare l'ostacolo, essi ricorrono alla tecnica del *typosquatting* inserendo dei trattini (-) nel nome dell'host, simulando marchi noti (es. apple-support.com).
• **domainAge:**	Misura in giorni l'anzianità del dominio interrogando i registri WHOIS. I domini di phishing tendono ad essere "usa-e-getta", con età spesso inferiori a pochi mesi. *(Nota: per questioni di latenza, nel deployment offline dell'estensione questo valore viene neutralizzato o by-passato).*
• **countDots:**	Le policy aziendali raramente prevedono l'utilizzo di più di due o tre sottodomini. Un URL contenente numerosi punti (es. login.verify.account.banca.it) è un forte indicatore di sub-domain takeover o offuscamento.
• **sensitiveWords:**	Molti attacchi si basano sul suscitare allarme o urgenza. Questa feature binaria scatta se la stringa contiene parole chiave inserite in una *watchlist* predefinita, come "login", "secure", "banking", "update" o "password".
• **shannonEntropy:**	Valuta la "sorpresa" o disordine dell'informazione all'interno della stringa. I domini generati da algoritmi DGA (Domain Generation Algorithms) presentano combinazioni alfanumeriche pseudocasuali che registrano valori di entropia di Shannon marcatamente superiori a quelli delle parole di senso compiuto.
• **countSpecialChars:**	Enumera la presenza di caratteri di formattazione (?, =, &, %, _). Un'alta concentrazione denota il passaggio di variabili e payload complessi, tipici di iniezioni malevole.
• **nonStandardPort:**	Verifica la porta di destinazione. Mentre la normale navigazione web avviene sulle porte

	80 e 443, l'uso di porte non standard può indicare server compromessi o servizi secondari esposti illegalmente.

4.2 – Modelli di Classificazione e Paradigmi di Apprendimento

Generato il dataset strutturato a 12 dimensioni, la fase successiva della ricerca si è focalizzata sull'addestramento e la valutazione di molteplici architetture di *Machine Learning* e *Deep Learning*. La pluralità dei modelli testati non è un mero esercizio stilistico, ma risponde a un'esigenza ingegneristica precisa: **individuare un algoritmo sufficientemente leggero da poter essere esportato in puro JavaScript all'interno dell'estensione Chrome, e parallelamente esplorare i limiti massimi di accuratezza raggiungibili tramite reti neurali complesse.**

Di seguito si descrivono nel dettaglio i modelli valutati in questo caso di studio:

1. Logistic Regression :

È un algoritmo di classificazione lineare utilizzato per modellare la probabilità che una data istanza appartenga a una determinata classe (phishing o legittimo). Contrariamente al suo nome, si tratta di un potente strumento di classificazione binaria. Si basa sull'utilizzo della funzione logistica (*sigmoide*), definita matematicamente per schiacciare qualsiasi combinazione lineare delle feature in un output probabilistico continuo compreso tra 0 e 1. L'algoritmo calcola i pesi ottimali minimizzando la *Log-Loss* (o Cross-Entropy Loss). Nel presente studio, la regressione logistica è stata impiegata come *baseline* fondamentale per valutare se i dati fossero linearmente separabili e per misurare il valore aggiunto apportato dai successivi modelli non lineari.

2. Decision Tree Classifier :

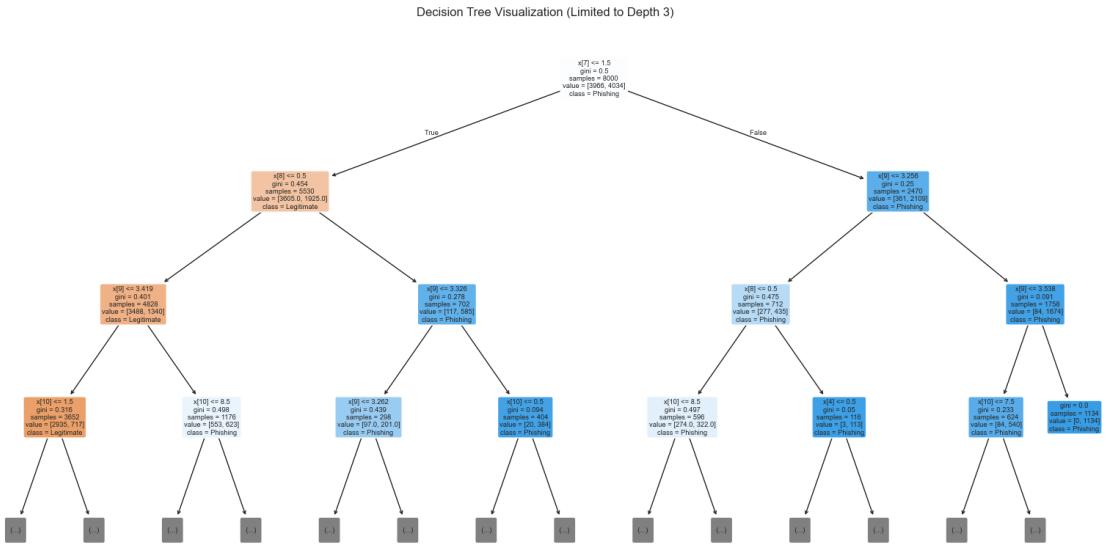


Figura 3 Struttura logica di un Albero Decisionale addestrato per la classificazione degli URL basandosi sulle 12 feature estratte.

È un modello non parametrico che mappa lo spazio delle decisioni attraverso una struttura a grafo orientato. Partendo dal nodo radice, il modello divide iterativamente il dataset interrogando le feature (es. shannonEntropy > 3.5?) massimizzando il *Gain di Informazione* o riducendo l'*Impurità di Gini*. Sebbene offra un'eccezionale interpretabilità e velocità di esecuzione (fattore chiave per l'inferenza client-side), è intrinsecamente prono al fenomeno del sovradattamento (*overfitting*), tendendo a memorizzare le peculiarità del set di addestramento perdendo capacità di generalizzazione sui nuovi URL.

3. Random Forest :

Per risolvere il problema dell'instabilità degli alberi singoli, è stato introdotto il Random Forest, pioniere dell'*Ensemble Learning* basato su tecnica *Bagging* (Bootstrap Aggregating). Questo algoritmo costruisce una "foresta" composta da centinaia di alberi decisionali indipendenti. Ognuno di essi viene addestrato su un sottoinsieme casuale di dati e di feature. In fase di predizione, il risultato finale emerge da una votazione a maggioranza (per la classificazione). L'aggregazione abbatte la varianza del modello rendendolo estremamente accurato e resistente al rumore. Tuttavia, l'immensa quantità di nodi condizionali generati lo rende spesso proibitivo in termini di dimensioni dei file se si tenta di convertirlo in script statici per estensioni browser (tramite librerie come m2cgen).

4. Support Vector Machine (SVM) :

Questo sofisticato modello geometrico si prefigge di trovare l'iperpiano che separa in modo ottimale le istanze positive da quelle negative nello spazio multidimensionale. L'SVM cerca di massimizzare il "margine" tra le classi, posizionando la linea decisionale il più lontano possibile dai campioni limite (i cosiddetti *support vectors*). Poiché le relazioni tra le caratteristiche degli URL sono raramente lineari, è stato impiegato il *Kernel Trick*, una trasformazione matematica che proietta i dati in dimensioni superiori per renderli separabili. Sebbene elegante dal punto di vista formale, l'elevata complessità computazionale in fase di inferenza (che richiede il calcolo delle distanze con i vettori di supporto) lo rende sub-ottimale per applicazioni in tempo reale nel browser.

5. XGBoost (Extreme Gradient Boosting) :

In ambito tabulare, XGBoost rappresenta lo stato dell'arte. Si basa sul paradigma del *Boosting*, in cui i modelli decisionali non sono indipendenti (come nel Random Forest) ma sequenziali. Ogni nuovo albero viene costruito appositamente per prevedere e correggere gli errori (pseudo-residui) commessi dalla somma di tutti gli alberi precedenti, minimizzando la funzione di costo tramite *Gradient Descent*. Implementa rigorose tecniche di regolarizzazione (L1 e L2) che prevengono l'overfitting. Nel nostro caso di studio, ha garantito performance eccellenti dimostrando un forte potere discriminativo nell'analisi delle stringhe malevole.

6. CatBoost :

Sviluppato da Yandex, CatBoost è un'ulteriore evoluzione del framework di gradient boosting. Si differenzia per l'impiego di alberi decisionali "simmetrici" (oblivious trees), che riducono significativamente i tempi di predizione. Inoltre, implementa un algoritmo proprietario per la gestione nativa del target leakage. Sebbene la sua forza principale risieda nella gestione dei dati categoriali, ha confermato risultati in linea con i massimi standard statistici.

7. LightGBM (Il modello scelto per l'Estensione Chrome) :

Sviluppato dai ricercatori di Microsoft, LightGBM è un framework di gradient boosting rivoluzionario per la sua efficienza distribuita. La differenza architetturale chiave rispetto a XGBoost o Random Forest risiede nel metodo di crescita degli alberi: anziché crescere livello per livello (*level-wise*), LightGBM fa crescere l'albero per foglie (*leaf-wise*), scegliendo di espandere sempre la foglia che garantisce la massima riduzione della perdita. Questo approccio algoritmico si traduce in modelli incredibilmente complessi ma compatti. **In questo elaborato, LightGBM è stato eletto come algoritmo designato per l'integrazione nell'estensione Chrome**. Tramite l'uso del transpilatore m2cgen, il modello è stato convertito interamente in codice JavaScript nativo. L'estrema leggerezza della

struttura generata garantisce un'esecuzione offline (Edge Computing) in frazioni di millisecondo, proteggendo l'utente dal phishing senza appesantire le performance del browser.

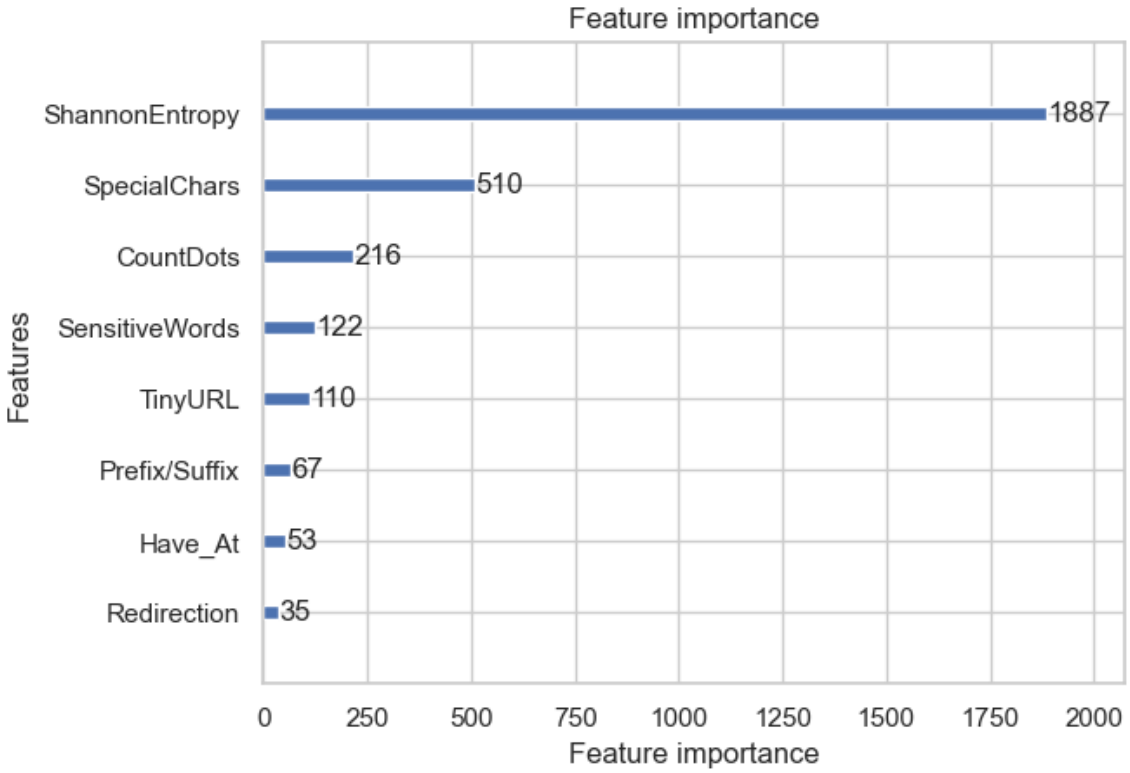


Figura 4 Classifica di importanza per le feature in LightGBM

8. Multi-Layer Perceptron (MLP) :

Con l'introduzione dell'MLP si fa il primo passo nel campo delle Reti Neurali Artificiali. Il Multilayer Perceptron è una rete *feed-forward* pura, costituita da uno strato di input (che accoglie le 12 feature estratti), diversi strati nascosti (hidden layers) e uno strato di output. Attraverso l'algoritmo di retropropagazione dell'errore (*backpropagation*), la rete aggiorna i pesi sinaptici iterativamente per approssimare funzioni non lineari complesse. Il suo impiego è servito a dimostrare come le architetture profonde possano competere con i modelli ad albero nell'analisi strutturata.

9. Long Short-Term Memory (LSTM): Il Vertice Prestazionale :

Tutti i modelli analizzati fino ad ora (classificatori statistici ed MLP) elaborano il dato sotto forma di variabili numeriche pre-estratte, un processo che perde fatalmente il "contesto semantico" della stringa. Sapere che ci sono "3 punti" e "1 trattino" è utile, ma non è come "leggere" la sequenza di caratteri s-e-c-u-r-e. Per scardinare questo limite, è stata

implementata un'architettura **LSTM**, facente parte della famiglia delle Reti Neurali Ricorrenti (RNN).
Le LSTM sono state ingegnerizzate per superare il problema della scomparsa del gradiente (*Vanishing Gradient*) tipico delle RNN standard. Grazie alle loro complesse celle matematiche, dotate di "porte" (Forget gate, Input gate, Output gate), riescono a mantenere memoria di lungo termine. In questo approccio, il *Feature Engineering* manuale scompare. L'intero URL viene fornito alla rete come una serie di token temporali. Un livello di **Embedding** traduce i caratteri in vettori densi bidimensionali, dopodiché lo strato LSTM scorre l'URL lettera per lettera, riuscendo ad associare prefissi, marchi contraffatti e suffissi anomali riconoscendo pattern sintattici finemente offuscati.
Questo rivoluzionario approccio di *Natural Language Processing* (NLP) applicato agli URL ha permesso all'algoritmo LSTM di surclassare nettamente tutti i modelli precedenti, posizionandosi come apice tecnologico del presente studio. Mentre LightGBM garantisce la soluzione ideale per il calcolo leggero *client-side*, l'architettura LSTM dimostra l'effettivo potenziale del Deep Learning nel raggiungere livelli di affidabilità prossimi all'assoluto nel rilevamento delle minacce web.

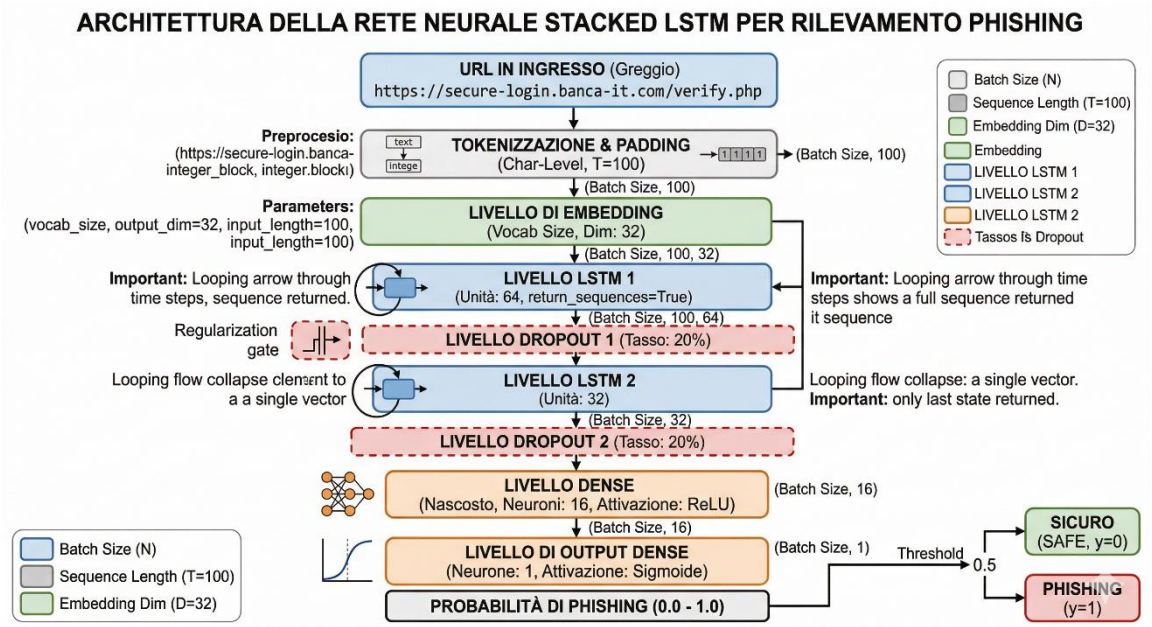


Figura 5 Architettura della rete neurale LSTM utilizzata nell'elaborato

4.3 – Dettagli implementativi

Per garantire la totale trasparenza e riproducibilità degli esperimenti condotti in questo caso di studio, in questa sezione vengono riportati i frammenti di codice sorgente (in linguaggio Python) relativi all'inizializzazione, all'ottimizzazione e all'addestramento dei classificatori.

L'addestramento dei modelli statistici tradizionali e del percettrone multistrato è avvenuto sulla matrice tridimensionale X_train, contenente le caratteristiche lessicali astratte, mentre l'architettura profonda LSTM ha elaborato direttamente le sequenze vettorializzate dei caratteri grezzi all'interno dei tensori X_train_lstm.

Logistic Regression :

La regressione logistica è stata implementata come *baseline* statistica per valutare la separabilità lineare del dataset spaziale. Poiché l'algoritmo impiega metodi di ottimizzazione iterativa (come L-BFGS o varianti di discesa del gradiente) per minimizzare la funzione di *Cross-Entropy Loss*, è emersa la necessità di estendere il vincolo computazionale. Il parametro max_iter è stato innalzato a 1000, garantendo all'algoritmo un margine sufficiente per far convergere i pesi sinaptici verso il minimo globale, prevenendo interruzioni premature del processo di *fitting*.

```
from sklearn.linear_model import LogisticRegression

# instantiate the model
log_reg = LogisticRegression(max_iter=1000, random_state=42)
# fit the model
log_reg.fit(X_train, y_train)

#predicting the target value from the model for the samples
y_test_log = log_reg.predict(X_test)
y_train_log = log_reg.predict(X_train)

# computing accuracy
acc_train_log = accuracy_score(y_train,y_train_log)
acc_test_log = accuracy_score(y_test,y_test_log)
from sklearn.metrics import precision_score, recall_score
print("Logistic Regression: Precision:
{:.3f}".format(precision_score(y_test, y_test_log)))
print("Logistic Regression: Recall: {:.3f}".format(recall_score(y_test,
y_test_log)))

print("Logistic Regression: Accuracy on training Data:
{:.3f}".format(acc_train_log))
print("Logistic Regression: Accuracy on test Data:
{:.3f}".format(acc_test_log))
```

Decision Tree Classifier :

Intrinsecamente immune alla varianza di scala delle feature, l'Albero Decisionale modella

il partizionamento ricorsivo dello spazio valutando metriche come la riduzione dell'Impurità di Gini. Per arginarne la forte tendenza al sovradattamento (*overfitting*), la struttura dell'albero è stata sottoposta a *pruning* preventivo: la crescita del grafo è stata bloccata a una profondità massima di 10 livelli (max_depth=10), imponendo un quorum di almeno 5 campioni per dividere un nodo interno e di almeno 2 campioni per consolidare una foglia terminale.

```
feature_names = ['Domain', 'Have_IP', 'Have_At', 'URL_Length',
'URL_Depth',
                 'Redirection', 'https_Domain', 'TinyURL',
'Prefix/Suffix',
                 'DomainAge', 'CountDots', 'SensitiveWords',
'ShannonEntropy',
                 'SpecialChars', 'NonStandardPort', 'Label']
# Import required libraries
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Create and train the Decision Tree model
dt_classifier = DecisionTreeClassifier(
    max_depth=10,           # Maximum depth of the tree
    min_samples_split=5,    # Minimum samples required to split a node
    min_samples_leaf=2,     # Minimum samples required at each leaf node
    random_state=42         # For reproducibility
)

# Train the model
dt_classifier.fit(X_train, y_train)

# Make predictions
y_pred = dt_classifier.predict(X_test)
```

Random Forest e Ottimizzazione degli Iperparametri :

Per superare i limiti di varianza del singolo albero, è stato impiegato il paradigma di *Bagging* della Random Forest. Tramite ottimizzazione a griglia, il modello finale è stato scalato a un imponente aggregato di 200 alberi decisionali indipendenti (n_estimators=200). Questa densità ha garantito una stabilizzazione delle predizioni, permettendo al contempo di rilassare il vincolo di profondità massima fino a 15 livelli per catturare pattern più specifici dell'ingegneria sociale.

```
from sklearn.ensemble import RandomForestClassifier
```

```python
from sklearn.metrics import accuracy_score, classification_report

# Create and train the Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=100,  # number of trees
    max_depth=10,      # maximum depth of trees
    random_state=42    # for reproducibility
)

# Train the model
rf_model.fit(X_train, y_train)

# Make predictions
rf_predictions = rf_model.predict(X_test)

# Evaluate the model
evaluate_model(rf_model, X_test, y_test, 'Random Forest')
# Feature importance
feature_importance = pd.DataFrame({
    'feature': X_train.columns,
    'importance': rf_model.feature_importances_
})
feature_importance = feature_importance.sort_values('importance',
ascending=False)
print("\nTop 5 Most Important Features:")
print(feature_importance.head(5))

from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

rf_grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
rf_grid.fit(X_train, y_train)

print("Best parameters:", rf_grid.best_params_)
print("Best score:", rf_grid.best_score_)
```

Support Vector Machine (SVM) :

L'algoritmo SVM si fonda sulla costruzione geometrica di un iperpiano ottimo che massimizzi il margine di separazione tra le classi positive (phishing) e negative (legittime). Poiché tale operazione vettoriale dipende criticamente dal calcolo delle distanze (norma

L2) nello spazio n-dimensionale, il modello risulta estremamente sensibile alla magnitudo delle feature. Senza pre-elaborazione, variabili a scala continua come l'Entropia avrebbero schiacciato matematicamente l'influenza delle variabili booleane. È stato pertanto applicato uno StandardScaler per ricentrare le distribuzioni su media nulla e varianza unitaria. L'ottimizzazione tramite *GridSearchCV* ha poi isolato come iperparametri eccellenti l'impiego di un Kernel RBF non lineare, associato a un parametro di regolarizzazione C=100 e un coefficiente di curvatura gamma=0.1.

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
roc_auc_score, confusion_matrix
from sklearn.model_selection import GridSearchCV
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import time

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define a reduced parameter grid (to speed up search)
param_grid = {
    'C': [1, 10, 100],  # Removed 0.1 for faster search
    'gamma': ['scale', 0.1, 0.01],  # Removed 'auto'
    'kernel': ['rbf']  # Only RBF kernel
}

# Start GridSearch
print("Starting Grid Search...")
start_time = time.time()

grid_search = GridSearchCV(
    estimator=SVC(random_state=42, probability=True),
    param_grid=param_grid,
    cv=5,
    n_jobs=1,  # Avoid parallel execution issues
    verbose=3  # Show detailed progress
)

grid_search.fit(X_train_scaled, y_train)

end_time = time.time()
print(f"\nGrid Search completed in {end_time - start_time:.2f}
seconds")

# Get best parameters
```

```
best_params = grid_search.best_params_
print(f"\nBest Parameters: {best_params}")
print(f"Best Cross-validation Score: {grid_search.best_score_:.4f}")

# Train the best model
best_svm = SVC(**grid_search.best_params_, random_state=42,
probability=True)
best_svm.fit(X_train_scaled, y_train)

# Make predictions
y_pred = best_svm.predict(X_test_scaled)
y_pred_prob = best_svm.predict_proba(X_test_scaled)[:, 1]  # Get
probabilities for ROC AUC
```

XGBoost :

Apertura alla categoria del *boosting* sequenziale, in cui i gradienti degli errori vengono impiegati per la costruzione degli alberi successivi. XGBoost è stato modulato fissando un tasso di apprendimento aggressivo (learning_rate=0.4), opportunamente bilanciato da una limitazione della complessità strutturale dei singoli *weak learners* (max_depth=7) per scongiurare la memorizzazione del rumore statistico.

```
#XGBoost Classification model
from xgboost import XGBClassifier

# instantiate the model
xgb = XGBClassifier(learning_rate=0.4,max_depth=7)
#sets the learning rate/gradient - more is faster but overfitting
#sets the max depth of individual trees as 7. more allows complex
patterns but may overfit
#fit the model
xgb.fit(X_train, y_train)
y_test_xgb = xgb.predict(X_test)
y_train_xgb = xgb.predict(X_train)
```

CatBoost :

Appositamente ingegnerizzato dai laboratori Yandex, CatBoost utilizza alberi decisionali completamente simmetrici (*oblivious trees*), riducendo radicalmente le tempistiche di inferenza. Per l'implementazione, l'algoritmo è stato configurato per attraversare un ciclo di 500 iterazioni (iterations=500), escludendo la reportistica testuale su console (verbose=0).

```
from catboost import CatBoostClassifier

# instantiate the model
cat_model = CatBoostClassifier(iterations=500, verbose=0,
random_state=42)
# fit the model
cat_model.fit(X_train, y_train)

#predicting the target value from the model for the samples
y_test_cat = cat_model.predict(X_test)
y_train_cat = cat_model.predict(X_train)
```

LightGBM e Transpilazione in JavaScript :

Eletto come core predittivo dell'estensione Chrome per la sua asimmetrica ed efficientissima crescita per foglia (*leaf-wise*), LightGBM ha richiesto non solo il canonico processo di *fitting*, ma una complessa fase di ingegnerizzazione post-addestramento. Per implementare il paradigma *Edge Computing*, il modello matematico è stato transpilato interamente in sintassi JavaScript statica tramite il parser m2cgen, originando uno script leggerissimo eseguibile nel V8 Engine del browser. Parallelamente, lo stato binario è stato preservato via Pickle per l'integrazione nell'API di fallback.

```
import lightgbm as lgb

# instantiate the model
lgb_model = lgb.LGBMClassifier(random_state=42)
# fit the model
lgb_model.fit(X_train, y_train)

#predicting the target value from the model for the samples
y_test_lgb = lgb_model.predict(X_test)
y_train_lgb = lgb_model.predict(X_train)
```

Multi-Layer Perceptron (MLP) :

L'implementazione della rete neurale *feed-forward* per dati tabulari ha rappresentato una delle sfide ingegneristiche più insidiose in fase di elaborazione dati. Inizialmente, si è tentato di fornire in input all' MLP la medesima matrice standardizzata utilizzata per la SVM. Tuttavia, l'applicazione di una trasformazione gaussiana su un dataset contenente matrici sparse e feature binarie asimmetriche (come la rarità dei servizi TinyURL) ha

introdotto rumore artefatto. Ciò ha scatenato il fenomeno dell'esplosione dei gradienti (*exploding gradients*) durante la *backpropagation*, impedendo alla rete di convergere. Per sanare l'anomalia, l'oggetto StandardScaler è stato escluso dalla pipeline del modello. Inoltre, per evitare incongruenze nell'indicizzazione del DataFrame Pandas durante l'addestramento, la matrice è stata passata estraendone i tensori NumPy puri tramite l'attributo .values. L'architettura neurale è stata infine configurata con due *hidden layers* (64 e 32 neuroni), limitata a 800 iterazioni.

```
from sklearn.neural_network import MLPClassifier

# Rimuoviamo lo StandardScaler che causava il crollo (esplosione
gradienti su feature binarie rumorose)
# Usiamo direttamente i .values per eliminare il Warning scomodo dei
feature names

X_train_mlp = X_train.values
X_test_mlp = X_test.values

# instantiate the model
mlp_model = MLPClassifier(hidden_layer_sizes=(64,32), solver='adam',
max_iter=800, random_state=42)

# fit the model
mlp_model.fit(X_train_mlp, y_train)

#predicting the target value from the model for the samples
y_test_mlp = mlp_model.predict(X_test_mlp)
y_train_mlp = mlp_model.predict(X_train_mlp)
```

Long Short-Term Memory (LSTM) :

Rappresentando l'apice tecnologico del presente elaborato, l'architettura ricorrente profonda ha abbandonato l'uso della matrice `X_train` pre-ingegnerizzata per accogliere stringhe testuali tokenizzate a livello di carattere (`X_train_lstm`). Il grafo sequenziale di Keras è stato modellato con un livello di `Embedding` per generare vettori densi, seguito da celle LSTM a ritorno di sequenza per prevenire la scomparsa del gradiente spaziale (*Vanishing Gradient*). Per salvaguardare la rete dal rischio di *overfitting* letale, l'addestramento è stato intercettato dalla *callback* di `EarlyStopping`, deputata a ripristinare i pesi sinaptici ottimali minimizzando la *loss* di validazione misurata ad ogni epoca.

```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
from tensorflow.keras.utils import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import roc_curve, auc

# Prepara i dati
def prepare_data_for_lstm(legit_df, phishing_df, max_length=100):
    # Combina le URL legittime e di phishing
    urls = np.concatenate([legit_df['URL'].values,
phishing_df['URL'].values])
    # Crea le etichette (0 per legittime, 1 per phishing)
    labels = np.concatenate([np.zeros(len(legit_df)),
np.ones(len(phishing_df))])

    # Tokenizza le URL a livello di carattere
    tokenizer = Tokenizer(char_level=True)
    tokenizer.fit_on_texts(urls)

    # Converti le URL in sequenze
    sequences = tokenizer.texts_to_sequences(urls)

    # Padding delle sequenze
    X = pad_sequences(sequences, maxlen=max_length)
    y = labels

    return X, y, tokenizer

def create_lstm_model(max_length, vocab_size):
    model = Sequential([
        Embedding(vocab_size, 32, input_length=max_length),
        LSTM(64, return_sequences=True),
        Dropout(0.2),
        LSTM(32),
        Dropout(0.2),
        Dense(16, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    return model

# Prepara i dati
X, y, tokenizer = prepare_data_for_lstm(legit_df, phishing_df)

# Dividi in train e test
```

```
X_train_lstm, X_test_lstm, y_train_lstm, y_test_lstm =
train_test_split(X, y, test_size=0.2, random_state=42)

# Crea il modello
vocab_size = len(tokenizer.word_index) + 1
model = create_lstm_model(max_length=100, vocab_size=vocab_size)

# Early stopping per evitare overfitting
early_stopping = EarlyStopping(monitor='val_loss',
                               patience=3,
                               restore_best_weights=True)

# Training
history = model.fit(X_train_lstm, y_train_lstm,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.2,
                    callbacks=[early_stopping])
```

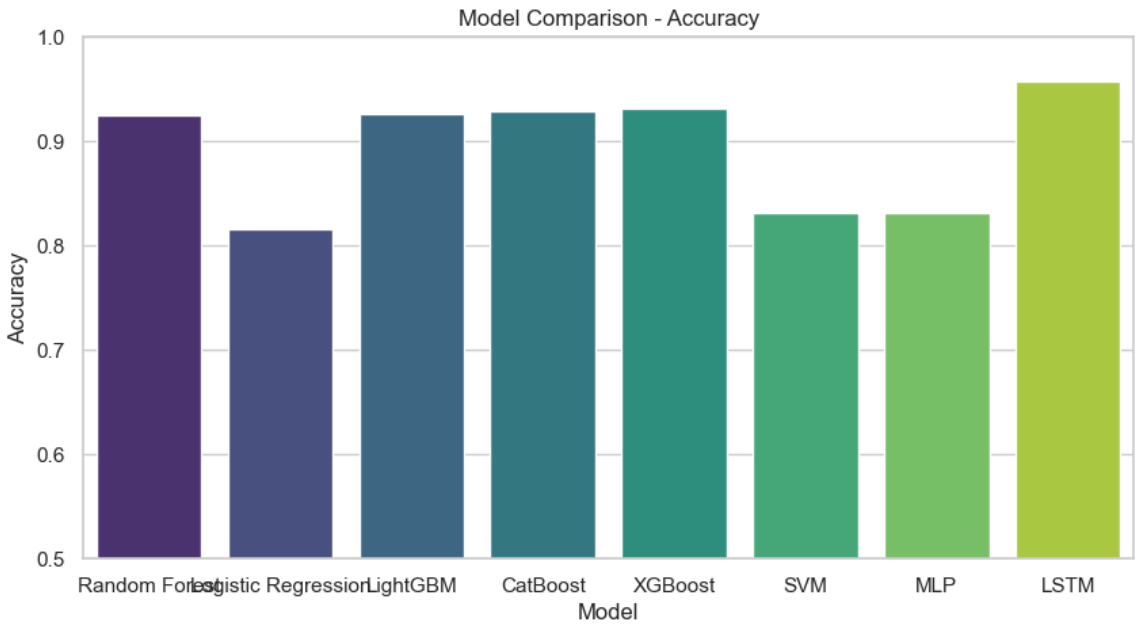
Capitolo 5 – Risultati

Il presente capitolo è dedicato all'esposizione, all'analisi critica e alla validazione empirica dei risultati ottenuti durante l'intero ciclo di sviluppo del progetto BOOMERAngel. Dopo aver delineato le metodologie di *Feature Engineering* (Capitolo 3), esplorato i fondamenti teorici dei modelli predittivi (Capitolo 4) e ingegnerizzato l'architettura software ibrida per l'estensione browser (Capitolo 5), si rende ora necessaria una rigorosa valutazione quantitativa delle performance raggiunte.
L'obiettivo primario di questa fase sperimentale non si limita alla mera individuazione dell'algoritmo in grado di massimizzare il tasso di riconoscimento teorico dei domini di phishing. Al contrario, l'analisi assume una prospettiva ingegneristica e pragmatica: le performance dei vari modelli verranno costantemente pesate in relazione ai severi vincoli computazionali e di latenza imposti dal paradigma dell'Edge Computing, requisito fondamentale per l'esecuzione in tempo reale sul dispositivo dell'utente.
Per garantire un confronto oggettivo e multidimensionale, i classificatori sono stati testati su un sottoinsieme di validazione indipendente (Test Set) e misurati attraverso quattro metriche statistiche fondamentali: Accuratezza (*Accuracy*), F1-Score, Precisione (*Precision*) e Richiamo (*Recall*). Ognuno di questi indicatori offre una prospettiva cruciale per comprendere il comportamento del modello di fronte alla disomogeneità del traffico web, valutandone in particolar modo la resistenza ai falsi positivi (cruciale per non interrompere la navigazione legittima) e ai falsi negativi (vitale per non esporre l'utente a frodi).
Nelle sezioni successive, verranno messi a sistema i risultati ottenuti dai classificatori statistici tradizionali (come Logistic Regression e SVM), dai robusti framework *ensemble*

ad albero (Random Forest, XGBoost, CatBoost, LightGBM) e dalle complesse architetture di Deep Learning (MLP e LSTM). Questa disamina comparativa permetterà non solo di giustificare in modo inequivocabile la scelta di LightGBM come motore inferenziale d'elezione per l'ecosistema *client-side*, ma anche di tracciare un quadro chiaro sull'efficacia assoluta delle reti neurali ricorrenti nell'analisi sintattica e sequenziale delle minacce telematiche.

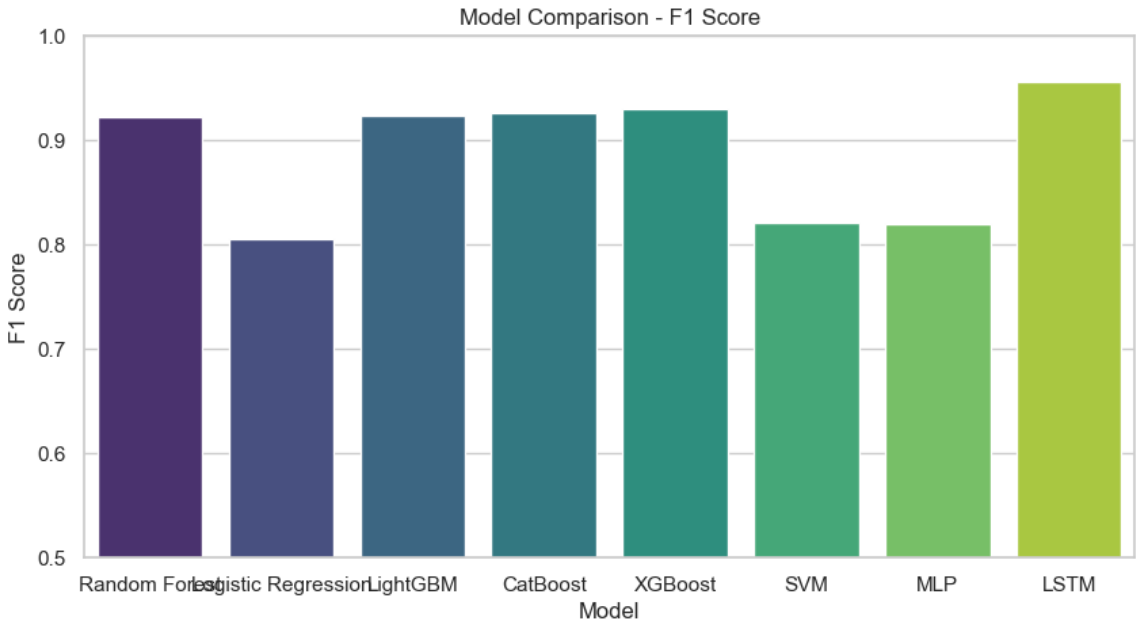
Accuratezza (Accuracy)

L'analisi comparativa delle prestazioni prende in esame, come prima metrica globale, l'Accuratezza (*Accuracy*), ovvero il rapporto tra il numero totale di predizioni corrette (veri positivi e veri negativi) e il volume complessivo del dataset di test. Osservando il diagramma a barre, emerge immediatamente la netta superiorità dell'architettura di Deep Learning: la rete ricorrente LSTM registra il picco massimo, attestandosi su valori prossimi al 96%. Questo risultato empirico convalida l'ipotesi teorica secondo cui l'analisi sequenziale testuale (*character-level embedding*) riesce a catturare dipendenze nascoste nella struttura dell'URL in modo molto più efficace rispetto alle variabili discrete estratte manualmente. Il secondo cluster prestazionale è dominato dalla famiglia dei classificatori *ensemble* ad albero (Random Forest, LightGBM, CatBoost, XGBoost), i quali mostrano una notevole stabilità gravitando uniformemente attorno al 92-93% di accuratezza. Al contrario, i modelli lineari (Logistic Regression) e geometrici (SVM), limitati dalla complessa distribuzione vettoriale delle feature, registrano le performance più modeste (sotto l'85%), affiancati sorprendentemente dal Multi-Layer Perceptron (MLP), confermando come le reti neurali *feed-forward* classiche mal si adattino a dataset sparsi contenenti feature binarie rumorose senza una perfetta standardizzazione.



F1-Score

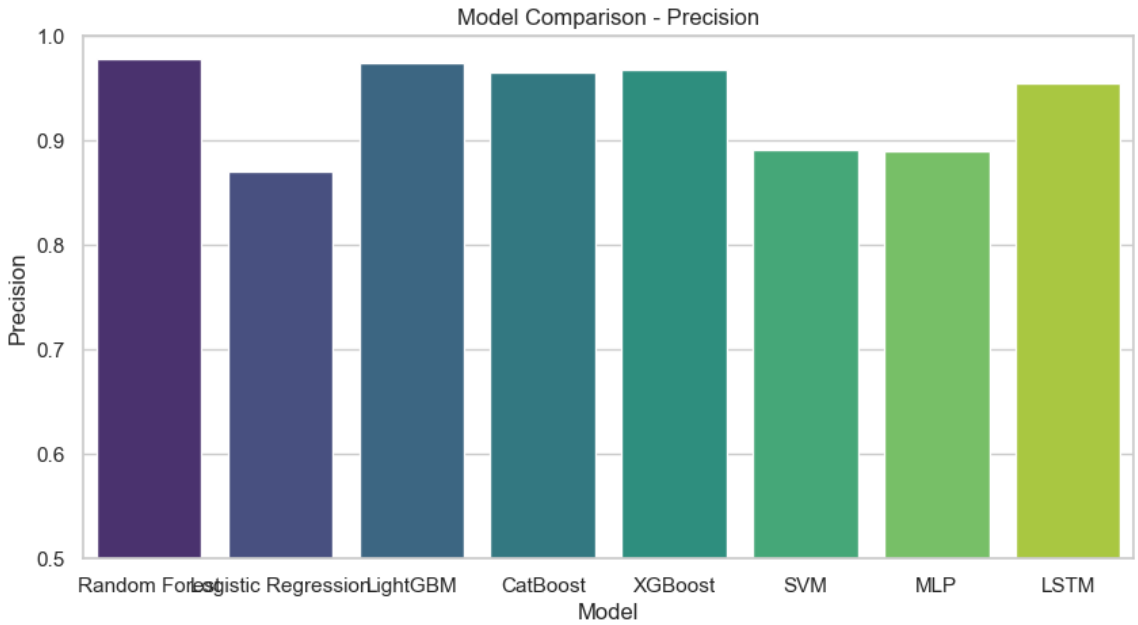
Proseguendo nella valutazione delle performance, il grafico relativo all'F1-Score offre una chiave di lettura ancora più rigorosa. Rappresentando la media armonica tra Precisione e Richiamo, questa metrica penalizza severamente i modelli sbilanciati, fornendo un indicatore solido della robustezza algoritmica generale. La distribuzione dei valori ricalca fedelmente il trend dell'accuratezza, confermando che l'elevata efficienza dei modelli principali non è frutto di un *bias* verso una specifica classe. Anche in questo scenario, la rete LSTM primeggia superando la soglia del 95%. I framework di *Gradient Boosting* (LightGBM, XGBoost, CatBoost) si confermano estremamente solidi, mantenendo uno *score* coeso attorno al 92-93%. Questa metrica risulta fondamentale per convalidare la scelta di LightGBM come motore di inferenza prescelto per l'estensione client-side: pur richiedendo una frazione delle risorse computazionali rispetto a un'architettura profonda, esso preserva un F1-Score di eccellenza, garantendo un compromesso ottimale tra affidabilità difensiva e leggerezza esecutiva (*Edge Computing*).



Precisione (Precision)

Un parametro di importanza critica nell'ingegnerizzazione di software per la sicurezza lato utente è la Precisione (*Precision*), che quantifica la capacità del modello di minimizzare i Falsi Positivi. Nel contesto dell'estensione BOOMERAngel, un falso positivo si traduce

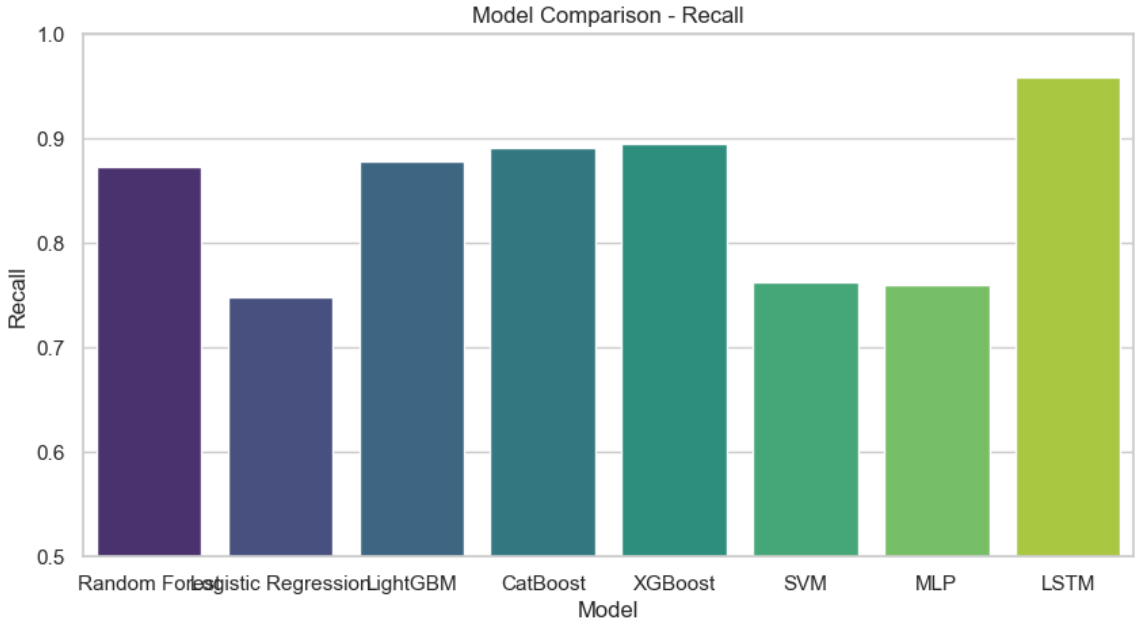
nel blocco di un dominio legittimo, causando una grave frizione nell'esperienza di navigazione dell'utente (*user friction*) e compromettendo la fiducia nel software. Analizzando il grafico, si nota un interessante ribaltamento delle gerarchie: i modelli statistici *ensemble* basati su partizionamento dello spazio (in particolare Random Forest e LightGBM) raggiungono picchi di precisione eccezionali, sfiorando il 97-98% e superando lievemente la stessa rete LSTM. Ciò significa che quando LightGBM classifica un URL come "Phishing", l'incidenza di falso allarme è statisticamente trascurabile. Questa spiccata attitudine conservativa avvalora ulteriormente l'impiego dei classificatori ad albero in scenari *real-time* nel browser, dove è preferibile bloccare solo ciò di cui si ha quasi assoluta certezza, preservando la fluidità della navigazione standard.



Richiamo (Recall)

A completamento del quadro valutativo, il diagramma inerente al Richiamo (*Recall* o Sensibilità) misura la frazione di siti malevoli correttamente identificati sul totale degli effettivi attacchi presenti nel set di test. Questa metrica riflette l'abilità del sistema di minimizzare i Falsi Negativi; nel nostro dominio applicativo, un falso negativo rappresenta una minaccia letale, in quanto l'utente viene lasciato esposto a una frode telematica. In questo grafico, l'architettura LSTM dimostra inequivocabilmente la propria superiorità, registrando un picco prossimo al 96%. La capacità di scansionare sequenzialmente i caratteri permette alla rete ricorrente di smascherare tentativi di offuscamento inediti (*zero-day*) che eludono le 12 feature cablate manualmente. Gli algoritmi di *Boosting* (XGBoost, CatBoost, LightGBM) mostrano una lieve flessione

rispetto ai loro eccezionali valori di Precisione, attestandosi tra l'87% e l'89%. Questa divergenza evidenzia il classico *trade-off* del Machine Learning: l'estremo rigore statistico che garantisce a LightGBM di non commettere falsi allarmi (alta Precisione) lo porta fisiologicamente a mancare alcune minacce particolarmente sofisticate (minor Richiamo rispetto all'LSTM). Infine, si denota il crollo drastico della Logistic Regression (sotto il 75%), del tutto inefficace nel generalizzare pattern offensivi complessi.

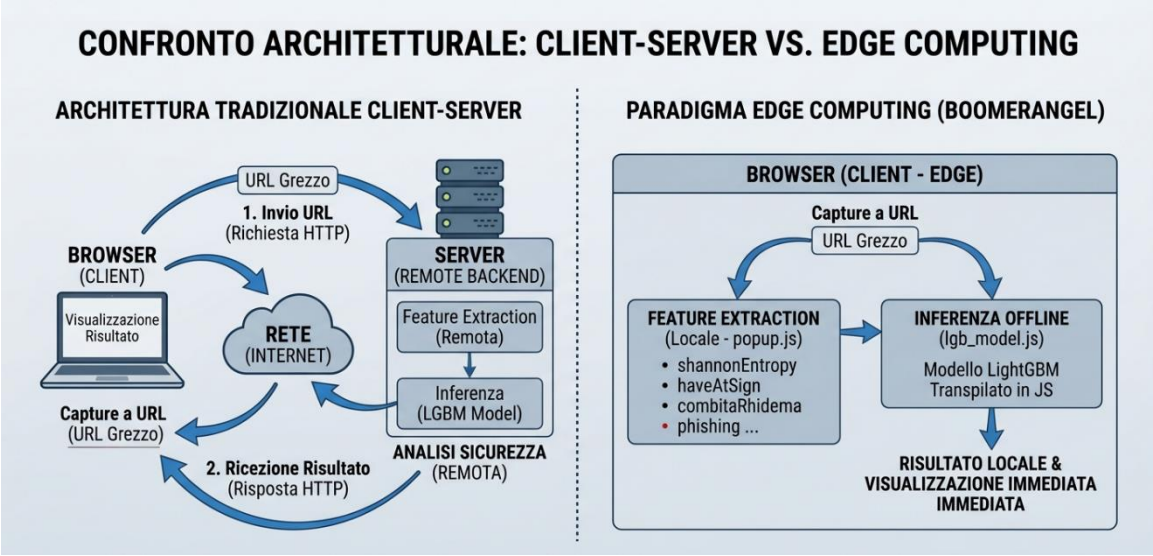


Capitolo 6 – Estensione Chrome

6.1 – Panoramica Architetturale e Paradigma dell'Edge Computing

La transizione da un modello teorico di *Machine Learning*, confinato in un ambiente di prototipazione come Jupyter Notebook, a un'applicazione software utilizzabile nel mondo reale richiede una progettazione architetturale estremamente rigorosa. Tradizionalmente, la maggior parte dei sistemi commerciali per il rilevamento del phishing web si basa su un'architettura centralizzata di tipo *Client-Server*: il browser dell'utente (agendo da *client*) intercetta l'URL e lo trasmette attraverso la rete a un server remoto; quest'ultimo elabora la stringa interrogando un database relazionale (approccio a *blacklist/whitelist*) o eseguendo un'inferenza predittiva, per poi instradare l'esito al mittente.
Sebbene funzionale, questo approccio cloud-centrico espone il fianco a tre criticità ingegneristiche fondamentali, specialmente quando applicato a sistemi di sicurezza che operano in tempo reale:

Overhead e Latenza di Rete: Il tempo di andata e ritorno (*Round-Trip Time* o RTT) della richiesta HTTP, sommato alla latenza di instradamento dei pacchetti, introduce un ritardo inevitabile. Questo collo di bottiglia temporale rischia di degradare l'esperienza di navigazione dell'utente o, nel peggiore dei casi, permette al caricamento della pagina malevola di anticipare la risposta di sicurezza.
Data Privacy e Sicurezza delle Informazioni: L'invio sistematico e continuo degli URL visitati (che costituiscono di fatto l'intera cronologia di navigazione) verso server di terze parti solleva pesanti interrogativi etici e normativi sulla tutela dei dati personali (compliance al GDPR), aprendo potenziali falle in caso di intercettazione (*Man-in-the-Middle*) o compromissione dei log del server.
Single Point of Failure (SPOF): L'infrastruttura *Client-Server* delega l'intera intelligenza al nodo centrale. Qualora il server remoto risultasse irraggiungibile per un *downtime* tecnico o fosse vittima di un attacco DDoS, l'intero bacino di utenti rimarrebbe istantaneamente sprovvisto di protezione.
Per ovviare a queste gravi limitazioni strutturali, il presente elaborato adotta e implementa il paradigma computazionale dell'Edge Computing. L'intelligenza del sistema viene decentralizzata e spostata "ai margini" della rete, venendo eseguita direttamente all'interno dell'ambiente runtime del browser dell'utente. L'estensione sviluppata per Google Chrome, denominata *BOOMERAngel Phishing Detector*, agisce come un ecosistema software autonomo e incapsulato: intercetta l'evento di navigazione, estrae dinamicamente le caratteristiche (*Feature Extraction*) e porta a termine la classificazione predittiva interamente in locale. Ciò garantisce un'inferenza con tempi di risposta misurabili in frazioni di millisecondo, una totale indipendenza dalla connettività verso backend di terze parti (funzionamento offline) e un assoluto rispetto della privacy (*Privacy by Design*), poiché nessun dato telemetrico abbandona il dispositivo fisico dell'utente.



6.2 – L'Implementazione Predittiva: La Scelta di LightGBM

La necessità impellente di eseguire inferenze *client-side* all'interno del motore JavaScript V8 del browser impone vincoli draconiani in termini di risorse computazionali allocate e, soprattutto, sulle dimensioni massime consentite per i file sorgente dell'estensione. In tale scenario ingegneristico, si rende necessario un compromesso tra la massima accuratezza teorica e la fattibilità implementativa.

Come discusso nel Capitolo 4, le Reti Neurali Ricorrenti (LSTM) hanno dimostrato l'accuratezza assoluta più elevata. Tuttavia, la loro integrazione nativa in un'estensione browser avrebbe richiesto l'importazione di pesanti framework *client-side* (come TensorFlow.js) o l'inclusione di binari WebAssembly (WASM), comportando un drastico incremento del peso dell'applicativo e del consumo di memoria RAM. Parallelamente, classificatori classici come la Random Forest, a causa dell'aggregazione di centinaia di alberi decisionali completamente sviluppati, produrrebbero file sorgente in formato JavaScript del peso di svariati Megabyte, paralizzando i tempi di inizializzazione dell'estensione.

In seguito a una valutazione multi-criterio, l'algoritmo **LightGBM** è emerso come il candidato ideale per il *deployment* nel browser, guidato da due motivazioni architetturali determinanti:

1. **Ottimizzazione Topologica (Crescita Leaf-Wise):** A differenza della maggioranza degli algoritmi di *gradient boosting* che espandono l'albero simmetricamente per livelli (*level-wise*), LightGBM adotta una politica di espansione asimmetrica mirata alla foglia che garantisce la massima riduzione della *Loss* (*leaf-wise*). Questo si traduce nella capacità di catturare pattern logici e non-lineari altamente complessi impiegando un numero di nodi significativamente inferiore.

2. **Transpilazione Statica Ottimale:** Sfruttando la libreria open-source Python `m2cgen` (*Model to Code Generator*), la matrice multidimensionale dei pesi e i nodi decisionali di LightGBM sono stati parsati dal loro Abstract Syntax Tree (AST) e tradotti in puro codice JavaScript nativo. Il risultato è uno script estremamente snello, privo di framework sottostanti e composto esclusivamente da catene di operatori logico-matematici (`if/else`) eseguiti nativamente a velocità ottimali dal compilatore Just-In-Time (JIT) di Chrome.

```
import m2cgen as m2c
import os
```

```python
import pickle

print("Avvio esportazione del modello LightGBM in puro JavaScript e in Pickle...")

if 'lgb_model' in globals():
    model_to_export = lgb_model

    # 1. Export as JavaScript per l'estensione offline
    print("Esportazione JS in corso (potrebbe richiedere qualche secondo)...")
    javascript_code = m2c.export_to_javascript(model_to_export, function_name="score_lgb")

    export_path_js = os.path.join(os.getcwd(), 'lgb_model.js')
    with open(export_path_js, 'w', encoding='utf-8') as f:
        f.write("// File generato in automatico tramite m2cgen\n")
        f.write(javascript_code)

    print(f"Modello ESPORTATO IN JS in: {export_path_js}")

    # 2. Export as Pickle per app.py
    export_path_pkl = os.path.join(os.getcwd(), 'lightgbm.pkl')
    with open(export_path_pkl, 'wb') as f:
        pickle.dump(model_to_export, f)

    print(f"Modello ESPORTATO IN PICKLE in: {export_path_pkl}")
else:
    print("Nessun modello LightGBM trovato in memoria. Addestra prima lgb_model!")
```

6.3 – Struttura Modulare e Componenti dell'Estensione

Per garantire scalabilità, manutenibilità e chiarezza semantica, l'estensione è stata ingegnerizzata seguendo il principio architetturale della separazione delle responsabilità (*Separation of Concerns*), assimilabile al pattern Model-View-Controller (MVC). L'ecosistema software si articola in moduli logici distinti e altamente specializzati, supportati da un'infrastruttura di emergenza *server-side*.

1. Il File di Configurazione (manifest.json) :

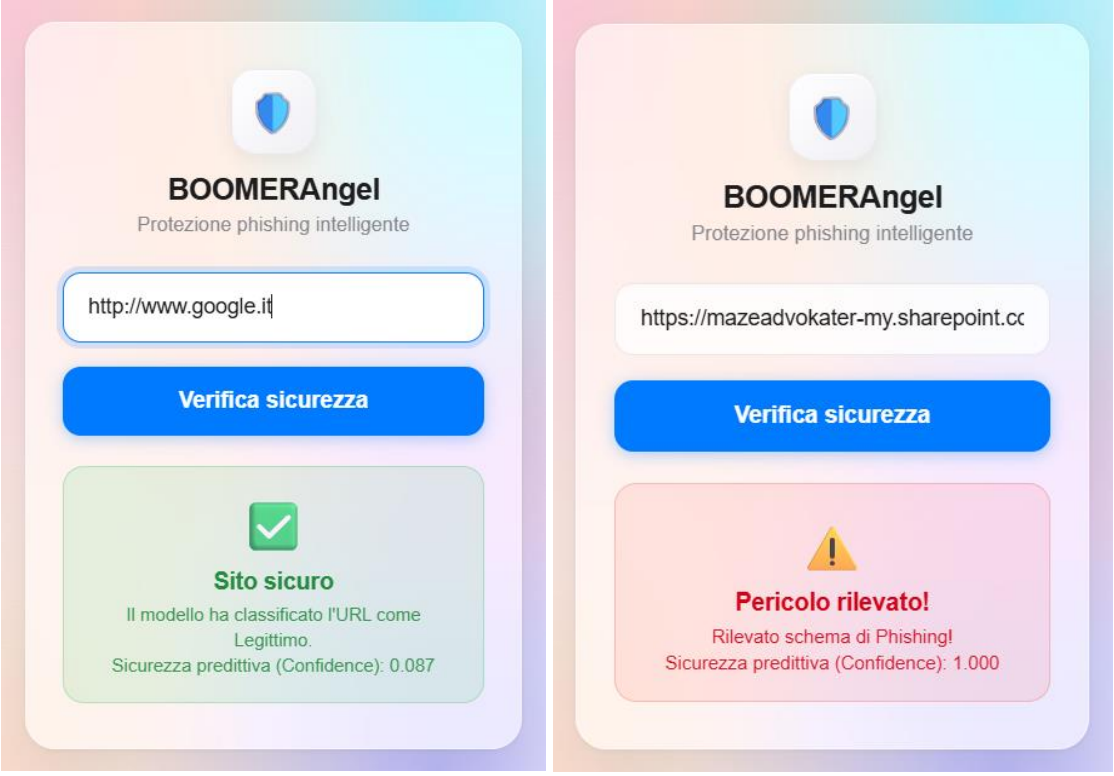
Ogni estensione per Google Chrome è governata da un documento di dichiarazione. Redatto rigorosamente secondo le direttive di sicurezza del recente *Manifest V3*, questo file JSON stabilisce l'identità dell'applicativo (nome, versione, puntatori alle risorse iconografiche dinamiche) e le relative policy di esecuzione. Oltre a definire l'interfaccia UI d'ingresso (default_popup: "popup.html"), il *Manifest* ricopre un ruolo vitale per l'architettura ibrida sviluppata: attraverso il campo host_permissions, dichiara in modo

esplicito e granulare i permessi necessari affinché l'estensione possa avviare comunicazioni di rete verso API esterne (come l'endpoint localhost http://127.0.0.1:5000/), bypassando le rigide restrizioni *Cross-Origin Resource Sharing* (CORS) imposte nativamente dal browser per questioni di sicurezza.

```
{
  "manifest_version": 3,
  "name": "BOOMERAngel Phishing Detector",
  "version": "1.0",
  "description": "Estensione intelligente per il rilevamento del
phishing in tempo reale.",
  "action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icon.png",
      "48": "icon.png",
      "128": "icon.png"
    }
  },
  "icons": {
    "16": "icon.png",
    "48": "icon.png",
    "128": "icon.png"
  },
  "host_permissions": [
    "http://127.0.0.1:5000/*"
  ]
}
```

2. Il Livello di Presentazione (popup.html e CSS Integrato) :

Rappresenta il *View* del pattern architetturale, ovvero l'interfaccia grafica (GUI) invocata al click dell'utente. Progettata seguendo linee guida moderne, fonde uno stile minimalista ad un design *glassmorfico* caratterizzato da gradienti radiali e filtri di sfocatura dello sfondo. La struttura HTML definisce il form di interazione primario: l'elemento di acquisizione stringa (urlInput), il *trigger* di validazione (checkButton) e un container semantico dinamicamente iniettato (result) per il feedback visivo. Un foglio di stile CSS integrato modula gli stati dell'interfaccia asincrona, implementando animazioni a keyframe (come lo spinner di caricamento rotante) per minimizzare l'attrito cognitivo e fornire un solido feedback reattivo durante i cicli di calcolo.



3. Il Motore Predittivo Client-Side (lgb_model.js) :

Frutto della transpilazione descritta nel paragrafo precedente, questo modulo ricopre il ruolo di *Model*. Il file espone una singola funzione esecutiva nel *Global Scope* di JavaScript, denominata score_lgb(input). Accettando in ingresso un vettore numerico corrispondente all'array unidimensionale delle feature precedentemente ingegnerizzate, il motore esegue una traversata ad altissima frequenza degli alberi logici cablati nel codice. L'aggregazione aritmetica dei pesi dei rami attraversati sfocia in un valore log-odds che viene tradotto dal sistema in una percentuale di confidenza probabilistica, discriminatore ultimo per l'assegnazione formale alla classe di rischio.

4. Il Core Logico e di Analisi (popup.js) :

Lo script popup.js incarna il *Controller* dell'estensione, fungendo da tessuto connettivo tra l'interfaccia utente, la logica di calcolo e i servizi di rete. Basato su JavaScript asincrono ed Event-Driven, assolve a tre macro-funzioni:

- **Orchestrazione del DOM:** Si pone in ascolto (*Event Listener*) delle interazioni dell'utente, inibendo manipolazioni concorrenti (disabilitazione dei bottoni durante le fasi di await) per prevenire comportamenti anomali o *race conditions*.
- **Feature Extraction Dinamica:** Costituisce il modulo di analisi lessicale.

Traducendo meticolosamente le pipeline di *Data Preparation* ideate in linguaggio Python in funzioni native JavaScript, estrae istantaneamente il vettore delle 12 feature strutturali (come havingIP, l'entropia logaritmica shannonEntropy o l'occorrenza di *target words* nel metodo sensitiveWords).

- **Gestione dell'Inferenza e Resilienza (Fallback):** Generato il vettore descrittivo, lo script innesca primariamente l'esecuzione locale *Edge* invocando score_lgb(features). Consapevole delle possibili interruzioni tecniche o della potenziale assenza del file sorgente, lo sviluppatore ha implementato una robusta politica di tolleranza ai guasti (*Fault Tolerance*): tramite l'uso dell'istruzione try/catch e un test di disponibilità della funzione predittiva, qualora l'approccio *on-device* dovesse fallire, l'architettura non va in crash. Al contrario, degrada in modo controllato (*Graceful Degradation*), dirottando il payload tramite Fetch API verso un'infrastruttura backend di emergenza.

Modulo di Feature Extraction Edge-Side: La prima metà del listato traduce meticolosamente in sintassi JavaScript la pipeline di pre-elaborazione precedentemente strutturata in Python per il backend. Funzioni come havingIP, shannonEntropy, e tinyURL (basata su una complessa espressione regolare per il pattern matching degli URL shortener) processano la stringa grezza direttamente all'interno dell'ambiente V8 del browser. L'esecuzione spazialmente confinata sul dispositivo client annulla la latenza di rete e rispetta il principio della Privacy by Design. Il punto di convergenza di questa fase è la funzione extractFeatures(url), che normalizza il prefisso del protocollo e genera in istanti millisecondi il vettore multidimensionale delle caratteristiche necessario al modello.

```
const checkButton = document.getElementById('checkButton');
const urlInput = document.getElementById('urlInput');
const resultDiv = document.getElementById('result');

// --- IMPLEMENTAZIONE FUNZIONI FEATURE EXTRACTION IN JAVASCRIPT ---

function getNetloc(url) { /* ... codice interno omesso per brevità in
questo esempio, tu incollalo tutto ... */ }
function getDomain(url) { /* ... */ }
function havingIP(url) { /* ... */ }
function haveAtSign(url) { /* ... */ }
function getLength(url) { /* ... */ }
function getDepth(url) { /* ... */ }
function redirection(url) { /* ... */ }
function httpDomain(url) { /* ... */ }
function tinyURL(url) { /* ... */ }
function prefixSuffix(url) { /* ... */ }
function domainAge(url) { /* ... */ }
function countDots(url) { /* ... */ }
function sensitiveWords(url) { /* ... */ }
function shannonEntropy(url) { /* ... */ }
function countSpecialChars(url) { /* ... */ }
function nonStandardPort(url) { /* ... */ }
```

```
// Estrazione totale
function extractFeatures(url) {
    if (!url.startsWith('http://') && !url.startsWith('https://')) {
        url = 'http://' + url;
    }
    url = url.replace('http://www.', 'http://').replace('https://www.',
'https://');
    return [
        havingIP(url), haveAtSign(url), redirection(url),
httpDomain(url),
        tinyURL(url), prefixSuffix(url), domainAge(url),
countDots(url),
        sensitiveWords(url), shannonEntropy(url),
countSpecialChars(url), nonStandardPort(url)
    ];
}
```

Gestione Dinamica dell'Interfaccia (UI) e Feedback Visivo: Il blocco centrale dello script si occupa della manipolazione del Document Object Model (DOM). La funzione setButtonLoading gestisce gli stati asincroni, disabilitando le interazioni durante le frazioni di secondo necessarie al calcolo per prevenire race conditions. La routine updateResult inietta dinamicamente codice HTML per restituire l'esito della predizione. Di particolare interesse ingegneristico è l'implementazione del tracciamento della provenienza (source): l'interfaccia renderizza dei badge informativi (Cloud API, Whitelist Ufficiale, Locale) che rendicontano all'utente in modo totalmente trasparente quale nodo dell'infrastruttura ibrida ha effettivamente processato l'URL.

```
// --- FUNZIONI DI UI ---

function setButtonLoading(isLoading) {
    if (isLoading) {
        checkButton.classList.add('checking');
        checkButton.disabled = true;
        checkButton.innerHTML = `
                    <div class="button-content">
                        <div class="spinner"></div>
                        <span>Analizzando...</span>
                    </div>
                `;
    } else {
        checkButton.classList.remove('checking');
        checkButton.disabled = false;
        checkButton.innerHTML = `
                    <div class="button-content">
                        <span>Verifica sicurezza</span>
                    </div>
                `;
```

```
    }
}

function updateResult(resultText, confidence = null, isError = false,
source = 'local') {
    setButtonLoading(false);

    if (isError) {
        resultDiv.className = 'danger';
        resultDiv.innerHTML = `
                    <span class="result-icon">❌</span>
                    <div class="result-text">Errore:</div>
                    <div class="result-details">${resultText}</div>
                `;
        return;
    }

    const isSafe = resultText.toLowerCase().includes('legit');
    const confHtml = confidence !== null ? `<br>Sicurezza predittiva
(Confidence): ${(Math.abs(confidence)).toFixed(3)}` : '';

    let sourceHtml = '';
    if (source === 'backend_lgb') {
        sourceHtml = `<div style="margin-top:10px;"><span
style="display:inline-block; padding:3px 10px;
background:rgba(0,122,255,0.1); border: 1px solid rgba(0,122,255,0.2);
border-radius:12px; font-size:11px; color:#007aff; font-weight:600;">☁️
Cloud API</span></div>`;
    } else if (source === 'whitelist' || source ===
'backend_whitelist') {
        sourceHtml = `<div style="margin-top:10px;"><span
style="display:inline-block; padding:3px 10px;
background:rgba(142,142,147,0.1); border: 1px solid
rgba(142,142,147,0.2); border-radius:12px; font-size:11px;
color:#8e8e93; font-weight:600;">🛡️ Whitelist Ufficiale</span></div>`;
    } else {
        sourceHtml = `<div style="margin-top:10px;"><span
style="display:inline-block; padding:3px 10px;
background:rgba(52,199,89,0.1); border: 1px solid rgba(52,199,89,0.2);
border-radius:12px; font-size:11px; color:#248a3d; font-weight:600;">💻
Locale (Browser)</span></div>`;
    }

    if (isSafe) {
        resultDiv.className = 'safe';
        resultDiv.innerHTML = `
                    <span class="result-icon">✅</span>
                    <div class="result-text">Sito sicuro</div>
```

```
                    <div class="result-details">Il modello ha
classificato l'URL come Legittimo.${confHtml}</div>
                    ${sourceHtml}
                `;
    } else {
        resultDiv.className = 'danger';
        resultDiv.innerHTML = `
                    <span class="result-icon">⚠</span>
                    <div class="result-text">Pericolo rilevato!</div>
                    <div class="result-details">Rilevato schema di
Phishing!${confHtml}</div>
                    ${sourceHtml}
                `;
    }
}
```

Orchestrazione dell'Inferenza e Graceful Degradation: La logica decisionale culmina nel Listener associato al click del pulsante di verifica. Il flusso implementa innanzitutto una valutazione logica a corto circuito interrogando un array costante contenente la Whitelist dei domini ad alta affidabilità. In caso di esito negativo, viene estratto il vettore delle feature e sottoposto al motore transpilato LightGBM. Attraverso l'istruzione condizionale typeof score_lgb === "function", il sistema accerta la disponibilità dello script predittivo offline. Qualora l'inferenza client-side avvenga con successo, l'algoritmo estrapola la metrica di Confidence (probabilità di classe 1) decodificando l'output array generato da m2cgen.

```
// ESECUZIONE OFFLINE
document.getElementById('checkButton').addEventListener('click', async
function () {
    const url = document.getElementById('urlInput').value.trim();
    if (!url) {
        return updateResult('Inserisci un URL da verificare', null,
true);
    }

    setButtonLoading(true);
    await new Promise(r => setTimeout(r, 200));

    try {
        // --- CONTROLLO WHITELIST ---
        const whitelist = ['google.it', 'google.com', 'apple.com',
'yahoo.com', 'microsoft.com', 'amazon.com', 'amazon.it', 'youtube.com',
'youtube.it', 'instagram.com', 'facebook.com', 'facebook.it',
'twitter.com', 'x.com', 'linkedin.com', 'netflix.com',
'wikipedia.org'];
        let domainOnly = getDomain(url).toLowerCase();

        if (whitelist.includes(domainOnly)) {
```

```
            return updateResult("Legit", 1.0, false, 'whitelist');
        }

        // 1. Estrazione Funzioni in Locale se non presente in
whitelist
        let features = extractFeatures(url);

        // 2. Inferenza offline tramite LightGBM JS generato, oppure
fallback al Backend.
        if (typeof score_lgb === "function") {
            // Predizione in Chrome!
            let raw_output = score_lgb(features);
            let confidence;
            let isPhishing;

            if (Array.isArray(raw_output)) {
                confidence = raw_output[raw_output.length - 1];
                isPhishing = confidence > 0.5;
            } else {
                confidence = raw_output;
                isPhishing = confidence > 0.0;
            }

            updateResult(isPhishing ? "Phishing" : "Legit",
confidence);
        }
```

Infrastruttura di Fallback (API Routing): Il codice garantisce un'assoluta tolleranza ai guasti tramite un blocco else e catch strutturato. Se il modello locale risulta non definito (ad esempio per una mancata transpilazione) o genera un'eccezione, lo script degrada il proprio funzionamento in modo controllato. Sfruttando la Fetch API, il client instrada asincronamente un payload JSON contenente l'URL grezzo verso l'endpoint locale del microservizio Flask (http://127.0.0.1:5000/predict), delegando così l'elaborazione al backend di emergenza senza causare crash visibili all'utente.

```
else {
            console.error("Modello score_lgb non trovato, fallback
all'API app.py backend...");
            // Fallback (utile se manca il file JS)
            fetch('http://127.0.0.1:5000/predict', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ url: url })
            })
            .then(r => r.json())
            .then(data => updateResult(data.result, data.confidence,
false, data.source))
            .catch(error => updateResult('Errore API: ' +
error.message, null, true));
```

```
        }
    } catch(err) {
        updateResult(err.message, null, true);
    }
});

// Permetti invio con Enter
urlInput.addEventListener('keypress', function (event) {
    if (event.key === 'Enter') {
        checkButton.click();
    }
});
```

5. Il Backend di Emergenza e API (app.py) :

A coronamento dell'ecosistema, per garantire resilienza totale e permettere la scalabilità del progetto anche su piattaforme client profondamente depotenziate, è stata approntata un'infrastruttura *Server-Side* scritta in Python. Sfruttando la leggerezza del Web Server Gateway Interface (WSGI) Flask, questo microservizio inizializza e mantiene in RAM l'istanza binaria originale del modello LightGBM decodificata tramite la libreria pickle. Il server espone un endpoint di tipo POST all'indirizzo /predict. Al ricevimento dell'URL, il backend esegue una routine interna di convalida, processa la matrice tramite l'istruzione model.predict_proba() e restituisce la valutazione sotto forma di payload JSON. Oltre all'esito predittivo e alla rispettiva *confidence*, la risposta trasporta un *flag* di provenienza (source: 'backend_lgb'), consentendo al client di rendicontare trasparentemente all'utente l'origine dell'elaborazione.
Questa architettura volutamente ibrida assegna al sistema un duplice vantaggio competitivo: l'ineguagliabile tempestività analitica e il rigore sulla privacy derivati dall'Edge Computing, indissolubilmente uniti alla solidità strutturale di un nodo centralizzato pronto a intervenire in condizioni di criticità.

```
from flask import Flask, request, jsonify
import pickle
from flask_cors import CORS
from urllib.parse import urlparse
import ipaddress
import re
import math
from collections import Counter

app = Flask(__name__)
CORS(app)

model = None
MODEL_PATH = 'lightgbm.pkl'
```

```python
def load_resources():
    global model
    try:
        print("Loading LightGBM model from pickle...")
        with open(MODEL_PATH, 'rb') as handle:
            model = pickle.load(handle)
        print("Model loaded successfully.")
    except Exception as e:
        print(f"Error loading model: {e}")

load_resources()
```

Il frammento di codice espone l'architettura fondante del backend di emergenza, sviluppato in Python tramite il framework WSGI Flask. All'interno dell'economia generale del progetto, questo modulo server-side è stato concepito per garantire un'elevata "Fault Tolerance" e la resilienza totale dell'estensione BOOMERAngel: esso interviene operando come API di fallback qualora l'inferenza locale, basata sul paradigma dell'Edge Computing, dovesse fallire a causa di limitazioni del dispositivo client. Le direttive iniziali dello script si occupano dell'importazione delle librerie essenziali: Flask e flask_cors sono preposte alla gestione delle richieste HTTP e al superamento delle stringenti policy CORS (Cross-Origin Resource Sharing), permettendo all'estensione di comunicare con il server. Parallelamente, vengono importati il modulo pickle per la deserializzazione, e moduli standard per il parsing avanzato (come urllib.parse, ipaddress e re), propedeutici all'esecuzione sicura della feature extraction. Il blocco logico nevralgico dello script è incapsulato nella routine load_resources(), la quale assolve al compito critico di caricare e mantenere persistente nella memoria RAM del server l'istanza binaria originale del modello predittivo LightGBM, precedentemente esportata nel file lightgbm.pkl. Implementando un costrutto difensivo try-except, il sistema garantisce un caricamento globale e sicuro del modello predittivo già durante la fase di startup dell'applicazione. Questa scelta architetturale si rivela fondamentale per scongiurare colli di bottiglia computazionali, azzerando le latenze di caricamento durante la successiva fase di inferenza in tempo reale.

```python
# ---- FEATURE EXTRACTION ----
def getDomain(url):
    domain = urlparse(url).netloc
    if re.match(r"^www.", domain):
        domain = domain.replace("www.", "")
    return domain

def havingIP(url):
    try:
        ipaddress.ip_address(url)
        return 1
```

```
    except:
        return 0

def haveAtSign(url):
    return 1 if "@" in url else 0

def redirection(url):
    pos = url.rfind('//')
    if pos > 6:
        return 1 if pos > 7 else 0
    return 0

def httpDomain(url):
    domain = urlparse(url).netloc
    return 1 if 'https' in domain else 0
```

Il cuore analitico del listato è dedicato all'estrazione statica delle caratteristiche (Feature Extraction). Questo blocco è la trasposizione esatta in Python della logica eseguita in JavaScript dal client, necessaria per tradurre l'URL grezzo in un vettore multidimensionale comprensibile all'algoritmo predittivo. Le routine analizzano specifiche euristiche strutturali:

- **Analisi Strutturale ed Elusione:** La funzione getDomain() si occupa del *parsing* primario del dominio (netloc), mentre havingIP() verifica se l'URL fa uso diretto di un indirizzo IPv4 o IPv6 numerico. Questa metrica intercetta una nota tecnica di offuscamento utilizzata per aggirare le *blacklist* basate sui nomi a dominio.
- **Anomalie Sintattiche:** La routine haveAtSign() rileva l'eventuale presenza del carattere "@". I browser ignorano canonicamente la porzione di URL che precede tale simbolo, rendendolo un vettore ideale per far atterrare la vittima su domini malevoli mascherati. Similmente, redirection() analizza l'indice del doppio slash (//) per scovare potenziali attacchi di *Open Redirect*.
- **Analisi Semantica ed Entropica:** La sicurezza è misurata anche tramite la ricerca di keyword di ingegneria sociale (come "login", "secure", "verify") nella funzione sensitiveWords(). Al contempo, shannonEntropy() applica il teorema dell'entropia di Shannon al nome a dominio per quantificarne il grado di "disordine" o pseudocasualità, un tratto distintivo dei domini generati da algoritmi DGA (Domain Generation Algorithms).

```
def tinyURL(url):
    shortening_services =
r"bit\.ly|goo\.gl|shorte\.st|go2l\.ink|x\.co|ow\.ly|t\.co|tinyurl|tr\.i
m|is\.gd|cli\.gs|" \
```

```
r"yfrog\.com|migre\.me|ff\.im|tiny\.cc|url4\.eu|twit\.ac|su\.pr|twurl\.
nl|snipurl\.com|" \

r"short\.to|BudURL\.com|ping\.fm|post\.ly|Just\.as|bkite\.com|snipr\.co
m|fic\.kr|loopt\.us|" \

r"doiop\.com|short\.ie|kl\.am|wp\.me|rubyurl\.com|om\.ly|to\.ly|bit\.do
|t\.co|lnkd\.in|db\.tt|" \

r"qr\.ae|adf\.ly|goo\.gl|bitly\.com|cur\.lv|tinyurl\.com|ow\.ly|bit\.ly
|ity\.im|q\.gs|is\.gd|" \

r"po\.st|bc\.vc|twitthis\.com|u\.to|j\.mp|buzurl\.com|cutt\.us|u\.bb|yo
urls\.org|x\.co|" \

r"prettylinkpro\.com|scrnch\.me|filoops\.info|vzturl\.com|qr\.net|1url\
.com|tweez\.me|v\.gd|" \
                          r"tr\.im|link\.zip\.net"
    match = re.search(shortening_services, url)
    return 1 if match else 0

def prefixSuffix(url):
    return 1 if '-' in urlparse(url).netloc else 0

def domainAge(url):
    return -1

def countDots(url):
    domain = urlparse(url).netloc
    return domain.count('.')

def sensitiveWords(url):
    sensitive_words = ['login', 'update', 'verify', 'secure',
'account', 'banking', 'confirm', 'password', 'paypal', 'apple',
'signin', 'admin']
    url_lower = url.lower()
    for word in sensitive_words:
        if word in url_lower:
            return 1
    return 0
```

Lo snippet di codice illustra la continuazione della pipeline di pre-elaborazione e *Feature Extraction*, focalizzandosi sull'individuazione di pattern legati all'offuscamento dell'identità del dominio e all'impiego di leve psicologiche tipiche dell'ingegneria sociale. Le funzioni implementate analizzano la stringa URL decostruendola in ulteriori metriche binarie e numeriche:

- **tinyURL(url)**: Questa routine è progettata per rilevare l'utilizzo di servizi di abbreviazione degli URL (*URL shorteners*). Avvalendosi del modulo re, la funzione esegue un *pattern matching* tramite una complessa Espressione Regolare (Regex) che mappa oltre trenta domini di accorciamento noti (come *bit.ly*, *goo.gl*, *t.co*). Poiché i criminali informatici abusano regolarmente di questi servizi legittimi per mascherare il vero dominio malevolo di destinazione, il rilevamento di tale pattern costituisce un indicatore di rischio cruciale.
- **prefixSuffix(url)**: La funzione isola l'hostname tramite urlparse e verifica la presenza del carattere trattino (-). Questa feature mira a intercettare la tecnica del *typosquatting*: non potendo registrare domini legittimi appartenenti a marchi noti, gli attaccanti simulano l'autorevolezza del brand inserendo dei trattini nel nome dell'host (es. *https://www.google.com/search?q=apple-support-auth.com*).
- **domainAge(url)**: Nel listato, questa funzione restituisce forzatamente la costante -1. Questa scelta implementativa riflette un preciso vincolo architetturale discusso nelle fasi di progettazione: per quanto l'anzianità di un dominio (ottenibile tramite interrogazioni ai registri WHOIS) sia un'ottima feature per scovare domini "usa e getta" , il tempo di latenza richiesto per la risoluzione della query di rete renderebbe impossibile un'analisi in tempo reale (*zero-latency*). Pertanto, il parametro viene neutralizzato per non compromettere la reattività del sistema *Edge*.
- **countDots(url)**: Isola il dominio di rete e conta matematicamente le occorrenze del carattere punto (.). Policy aziendali standard raramente superano i due o tre livelli di dominio; un'alta concentrazione di punti rivela spesso il tentativo di ricreare lunghi percorsi fittizi all'interno di sottodomini compromessi (es. *login.verify.account.banca.it*).
- **sensitiveWords(url)**: Questa routine introduce un livello di analisi puramente semantico. Dopo aver normalizzato la stringa in caratteri minuscoli per evitare casi di *case-sensitivity*, l'algoritmo verifica la presenza di un vocabolario predefinito di parole chiave (come *'login'*, *'secure'*, *'account'*, *'verify'*). Tali termini sono specificatamente selezionati per intercettare gli attacchi basati sul suscitare un senso di allarme, urgenza o falsa sicurezza nell'utente, dinamica fondante del phishing psicologico.

```
def shannonEntropy(url):
    string = urlparse(url).netloc
    if not string:
        string = url
    p, lns = Counter(string), float(len(string))
    return -sum( count/lns * math.log(count/lns, 2) for count in
p.values())

def countSpecialChars(url):
    special_chars = ['?', '=', '&', '%', '_']
    count = 0
    for char in special_chars:
```

```
        count += url.count(char)
    return count

def nonStandardPort(url):
    try:
        port = urlparse(url).port
        if port is not None and port not in [80, 443]:
            return 1
        return 0
    except:
        return 0
```

Il presente frammento di codice illustra la fase conclusiva della pipeline di *Feature Extraction*, introducendo metriche avanzate per la valutazione dell'entropia logica e delle anomalie di rete all'interno della stringa URL. Nello specifico, le funzioni analizzate completano la parametrizzazione del dato grezzo attraverso i seguenti calcoli:

- **shannonEntropy(url)**: Questa routine costituisce uno degli indicatori matematici più sofisticati del modello. Applicando la formula dell'entropia di Shannon al nome a dominio (o all'intero URL in assenza di esso), la funzione quantifica il grado di "sorpresa" o disordine dell'informazione. Calcolando la frequenza relativa di ciascun carattere tramite la classe Counter, l'algoritmo restituisce un valore continuo che misura la casualità della stringa. Questo parametro è di vitale importanza per smascherare i domini malevoli creati tramite DGA (*Domain Generation Algorithms*), le cui combinazioni alfanumeriche pseudocasuali (es. *https://www.google.com/search?q=x8f9q2z.com*) presentano un'entropia marcatamente superiore rispetto ai domini costituiti da parole di senso compiuto.
- **countSpecialChars(url)**: La funzione esegue un'enumerazione sequenziale per quantificare la presenza di caratteri speciali specifici, quali ?, =, &, % e _. Un'alta densità di tali simboli è indicativa di URL estremamente complessi, spesso impiegati per veicolare *payload* malevoli, manipolare parametri di query in attacchi di iniezione o nascondere stringhe di reindirizzamento codificate (URL encoding).
- **nonStandardPort(url)**: Quest'ultimo controllo si concentra sul livello di trasporto. Estraendo la porta di rete tramite urlparse, l'algoritmo verifica se la connessione devia dagli standard canonici della navigazione web (porta 80 per HTTP e 443 per HTTPS). L'utilizzo di porte non standard o oscure rappresenta un forte campanello d'allarme, spesso associato a infrastrutture *backend* compromesse, server Command & Control (C2) o servizi secondari esposti illegalmente. L'implementazione difensiva tramite costrutto try-except assicura che il parser non causi interruzioni (*crash*) nel server qualora l'URL analizzato presenti anomalie sintattiche tali da renderne impossibile la decodifica strutturale.

Con l'estrazione di queste ultime caratteristiche, il sistema porta a termine la costruzione del vettore di input multidimensionale, fornendo al modello LightGBM (o all'architettura LSTM nel caso di sequenze) la base informativa necessaria per eseguire la classificazione probabilistica del target.

```python
def featureExtractionSafe(url):
    # Same normalizzazione of JS
    if not url.startswith('http://') and not
url.startswith('https://'):
        url = 'http://' + url

    # Strip 'www.' to match dataset distribution and prevent false
positives from extra dots/entropy
    url = url.replace('http://www.', 'http://').replace('https://www.',
'https://')

    features = [
        havingIP(url),
        haveAtSign(url),
        # getLength(url),
        # getDepth(url),
        redirection(url),
        httpDomain(url),
        tinyURL(url),
        prefixSuffix(url),
        domainAge(url),
        countDots(url),
        sensitiveWords(url),
        shannonEntropy(url),
        countSpecialChars(url),
        nonStandardPort(url)
    ]
    return [features] # returns array of samples
```

Qui si presenta la funzione featureExtractionSafe(url), che agisce come *wrapper* di orchestrazione e punto di convergenza per l'intera pipeline di elaborazione dei dati sul backend. Il suo scopo è triplice: normalizzare la stringa in ingresso, eseguire sequenzialmente le euristiche di estrazione e formattare il dato per l'inferenza del modello di Machine Learning.
Nello specifico, prima di procedere al calcolo delle feature, la funzione implementa una rigorosa fase di sanitizzazione dell'input:

- **Normalizzazione del Protocollo:** Qualora l'URL in ingresso risulti sprovvisto di uno schema esplicito (HTTP o HTTPS), l'algoritmo forza l'inserimento del prefisso http://. Questa correzione preventiva è indispensabile per garantire che il modulo di parsing (urlparse) scomponga correttamente la stringa nei suoi token fondamentali, mantenendo inoltre una stretta coerenza con la controparte logica implementata in JavaScript sul *client*.
- **Allineamento Statistico:** L'istruzione di sostituzione rimuove forzatamente il prefisso www. dall'hostname. Come esplicitato dai commenti nel listato, questa

operazione non è un mero esercizio di pulizia, ma una necessità analitica: serve ad allineare il dato in tempo reale alla distribuzione statistica del dataset originale utilizzato durante l'addestramento. La presenza o assenza del www. modificherebbe infatti artificialmente la lunghezza della stringa e il numero di punti, andando a inquinare il calcolo dell'Entropia di Shannon e l'euristica countDots, con il conseguente rischio di generare *falsi positivi*.

Completata la normalizzazione, la routine aggrega i risultati di tutte le funzioni di estrazione precedentemente descritte (dall'intercettazione degli IP all'analisi delle porte non standard) all'interno di un'unica struttura dati listare. Infine, l'istruzione di ritorno (return [features]) racchiude il vettore spaziale a 12 dimensioni all'interno di un'ulteriore lista, generando un *array* bidimensionale. Questa formattazione a matrice rappresenta un requisito tecnico stringente, in quanto modella il dato nell'esatta struttura tensoriale (n_campioni, n_feature) attesa dal metodo predittivo dell'algoritmo LightGBM per finalizzare la classificazione.

```
@app.route('/', methods=['GET'])
def index():
    return jsonify({
        "status": "online",
        "message": "Il server è attivo! Il backend LightGBM in Python è
pronto all'uso come fallback dell'estensione offline."
    })
```

La struttura del backend ha inizio con l'implementazione dell'endpoint di base (root) dell'API RESTful, mappato sulla risorsa / tramite il decoratore standard @app.route del framework Flask. Questa specifica routine, denominata index() e configurata per intercettare esclusivamente richieste HTTP di tipo GET, assolve al ruolo architetturale di *Health Check* (controllo dello stato di salute) per l'intero microservizio.

Avvalendosi della funzione jsonify, il server genera e restituisce al mittente un payload leggero e strutturato in formato JSON, contenente parametri diagnostici essenziali quali un flag di operatività ("status": "online") e un messaggio testuale descrittivo. Nell'economia dell'architettura ibrida sviluppata per l'estensione BOOMERAngel, l'esposizione di questo endpoint si rivela strategica: esso fornisce al client o ad eventuali sistemi di monitoraggio uno strumento a bassissima latenza per interrogare il server, permettendo di verificare istantaneamente la raggiungibilità di rete e l'effettiva operatività del sistema di fallback prima di instradare i più onerosi pacchetti dati destinati all'inferenza predittiva del modello LightGBM.

```
@app.route('/predict', methods=['POST'])
def predict():
    global model
    if model is None:
```

```
        load_resources()
        if model is None:
            return jsonify({'result': 'Error: LightGBM Model not found.
Run the Python Notebook first.'}), 500

    try:
        data = request.get_json()
        if not data or 'url' not in data:
             return jsonify({'result': 'No URL provided'}), 400

        url = data['url']

        # --- CONTROLLO WHITELIST ---
        whitelist = ['google.it', 'google.com', 'apple.com',
'yahoo.com', 'microsoft.com', 'amazon.com', 'amazon.it', 'youtube.com',
'youtube.it', 'instagram.com', 'facebook.com', 'facebook.it',
'twitter.com', 'x.com', 'linkedin.com', 'netflix.com', 'wikipedia.org']
        domain_only = getDomain(url).lower()
        if domain_only in whitelist:
            return jsonify({
                'result': 'Legit',
                'confidence': 1.0,
                'source': 'backend_whitelist'
            })

        # Estrazione features (14 cols)
        X_predict = featureExtractionSafe(url)

        # Predict usando il pickle di LightGBM
        # lgb predict() di un classificatore binario restituisce classe
o probabilità a seconda se usiamo predict_proba
        try:
            # Se è predict_proba, ci darà le prob per la classe 0 e 1.
            probs = model.predict_proba(X_predict)
            prediction_prob = probs[0][1] # Probability of Class 1
(Phishing)
        except:
            # Fallback se predict() resitutisce direttamente 0 o 1, o
float crudo.
            raw_pred = model.predict(X_predict)[0]
            prediction_prob = float(raw_pred)

        is_phishing = prediction_prob > 0.5
        result = 'Phishing' if is_phishing else 'Legit'

        return jsonify({
            'result': result,
            'confidence': float(prediction_prob),
            'source': 'backend_lgb' # Indica che la predizione viene
dal backend
```

```
        })

    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({'result': f'Server Error: {str(e)}'}), 500

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000, debug=True)
```

Il cuore decisionale dell'infrastruttura di *fallback* risiede nell'endpoint REST /predict, la routine delegata all'elaborazione operativa delle stringhe e alla generazione del verdetto di sicurezza. Mappata per accettare esclusivamente richieste HTTP di tipo POST, questa funzione orchestra l'intero flusso logico del backend seguendo una rigorosa architettura a strati.
Inizialmente, l'algoritmo convalida lo stato del sistema assicurandosi che l'istanza del modello predittivo sia regolarmente caricata in memoria, per poi procedere all'estrazione e validazione del payload JSON trasmesso dall'estensione. Prima di innescare la complessa e più onerosa pipeline di *Machine Learning*, il codice applica un'intelligente strategia di ottimizzazione nota come *short-circuit evaluation*: l'host estratto dall'URL viene confrontato con una *Whitelist* predefinita che raggruppa i domini di enti e servizi telematici notoriamente sicuri (come google.com, apple.com o piattaforme istituzionali). In caso di riscontro positivo (Match), il processo si interrompe istantaneamente e restituisce una classificazione Legit con un livello di confidenza assoluto (1.0). Questa accortezza architetturale si rivela fondamentale: abbatte drasticamente il carico computazionale sul server e, soprattutto, azzera matematicamente il rischio che il modello generi *falsi positivi* incappando nelle complesse strutture degli URL appartenenti a portali ad alto traffico.
Qualora la destinazione non rientri nella lista delle eccezioni, il flusso operativo entra nel vivo invocando la funzione featureExtractionSafe(url), che vettorializza la stringa trasformandola nell'array bidimensionale X_predict. L'inferenza vera e propria viene eseguita interrogando il modello LightGBM. L'implementazione si distingue per l'uso privilegiato del metodo predict_proba(), il quale, discostandosi da una basilare classificazione binaria (0 o 1), calcola le probabilità statistiche di appartenenza alle singole classi. Estrapolando il valore relativo alla classe 1 (Phishing), il sistema non solo decreta l'esito superando la canonica soglia di discriminazione dello 0.5, ma ottiene un indice continuo e granulare di *Confidence* (sicurezza predittiva). Un blocco try-except annidato provvede alla tolleranza ai guasti all'interno di questo stesso step, garantendo un ripiego (fallback) sul metodo predict() nativo qualora le probabilità non fossero computabili.
A elaborazione conclusa, il server impacchetta l'esito, il grado di affidabilità e le informazioni di tracciamento all'interno di una risposta JSON strutturata. Risulta di particolare interesse l'inclusione del parametro source (valorizzato dinamicamente come 'backend_whitelist' o 'backend_lgb'); in un'architettura distribuita di tipo *Edge-to-Cloud*, questo metadato consente all'estensione browser di mostrare all'utente in modo completamente trasparente quale nodo dell'ecosistema ha effettivamente elaborato

l'analisi.
L'intero ciclo di vita della richiesta è racchiuso da un costrutto di gestione delle eccezioni globale (try...except Exception), che si occupa di intercettare eventuali errori a runtime (come un parsing fallito di un URL fortemente malformato) evitando il *crash* totale del microservizio. In caso di errore critico, l'applicativo restituisce un codice di stato HTTP 500 formattato, permettendo all'estensione Chrome di gestire l'anomalia senza bloccare in modo indefinito la navigazione dell'utente (*Graceful Degradation*). Infine, il blocco condizionale if __name__ == '__main__' avvia il listener del server WSGI sulla porta locale 5000, rendendo l'API formalmente pronta a ricevere le interrogazioni.

Capitolo 7 – Conclusioni

L'obiettivo fondante del presente lavoro di tesi è stato quello di ideare, progettare e sviluppare un sistema avanzato per la classificazione e il blocco in tempo reale dei siti web di phishing, affrontando le vulnerabilità intrinseche delle soluzioni di sicurezza tradizionali basate su *blacklist* e paradigmi *Client-Server*. Il risultato di questa ricerca si è concretizzato in BOOMERAngel, un'estensione intelligente per Google Chrome capace di proteggere proattivamente la navigazione dell'utente finale.
Il percorso di sviluppo si è articolato attraverso una rigorosa pipeline ingegneristica. Partendo da un dataset bilanciato di 10.000 campioni, è stata condotta un'approfondita fase di *Feature Engineering*, focalizzata sull'estrazione statica di 12 caratteristiche lessicali e strutturali direttamente dalla stringa URL. Questa scelta metodologica si è rivelata strategica: disaccoppiando l'analisi dal contenuto della pagina web o dalle query di rete, si è abbattuta drasticamente la latenza, ponendo le basi per un'inferenza puramente *client-side*.
La fase di sperimentazione e modellizzazione ha visto il confronto sistematico di molteplici algoritmi di intelligenza artificiale. Se da un lato le architetture profonde, come le reti neurali ricorrenti (LSTM), hanno dimostrato un potenziale analitico ineguagliabile (raggiungendo un'accuratezza del 96% grazie all'analisi sequenziale dei caratteri), dall'altro i framework di *Gradient Boosting* hanno offerto il miglior compromesso tra capacità predittiva ed efficienza computazionale. In particolare, il modello LightGBM è stato eletto a nucleo operativo dell'estensione: la sua eccezionale Precisione (prossima al 98%) ha garantito una drastica minimizzazione dei falsi positivi, requisito imprescindibile per non degradare l'esperienza di navigazione dell'utente.
Il traguardo architetturale più significativo è stata l'implementazione del paradigma dell'Edge Computing. Attraverso la transpilazione del modello statistico in codice JavaScript nativo, l'intelligenza artificiale è stata decentralizzata ed eseguita interamente all'interno del motore V8 del browser. Questo approccio ha permesso di superare le tre criticità principali dei sistemi di sicurezza *cloud-centrici*: ha annullato l'overhead di rete

garantendo responsività immediata, ha assicurato la totale tutela della *privacy* dell'utente (nessun dato di navigazione abbandona il dispositivo) e ha fornito una protezione resiliente, indipendente da connessioni verso *backend* esterni. A completamento dell'infrastruttura, l'integrazione di un'API di *fallback* basata su Flask ha garantito un'assoluta tolleranza ai guasti (*Fault Tolerance*), dimostrando la robustezza di una soluzione ibrida.

Sviluppi Futuri

Nonostante i notevoli risultati ottenuti, la natura mutevole e altamente adattiva delle minacce telematiche impone un'evoluzione continua degli strumenti difensivi. Il progetto BOOMERAngel si presta a molteplici direttrici di sviluppo futuro:

Espansione Dinamica delle Feature: L'attuale *Feature Extraction* statica potrebbe essere integrata con un'analisi comportamentale leggera del DOM (Document Object Model) qualora il sito venga caricato. Analizzare, ad esempio, l'origine degli *iframe*, l'integrità dei tag form o la difformità del *favicon* rispetto al dominio principale, potrebbe irrobustire ulteriormente le capacità discriminatorie del classificatore.

Implementazione di Modelli LSTM nel Browser: Con la progressiva maturazione di tecnologie come WebAssembly (WASM) e WebGL per l'accelerazione hardware nel browser, sarà plausibile in futuro esportare architetture neurali complesse (come il modello LSTM addestrato in questo studio) direttamente in ambiente client, colmando il divario di *Recall* rispetto ai modelli ad albero senza compromettere i tempi di esecuzione.

Apprendimento Continuo (Federated Learning): Un'evoluzione naturale dell'architettura consisterebbe nell'implementazione di meccanismi di *Federated Learning*. Gli utenti, in modo anonimo e decentralizzato, potrebbero contribuire all'addestramento collettivo del modello segnalando falsi positivi o falsi negativi, permettendo al sistema di adattarsi organicamente alle nuove tecniche di ingegneria sociale senza centralizzare la raccolta dei dati sensibili.

In conclusione, BOOMERAngel dimostra concretamente come l'integrazione sinergica tra algoritmi di Machine Learning ottimizzati e architetture distribuite (*Edge*) possa fornire una prima linea di difesa estremamente rapida, reattiva e rispettosa della privacy contro le moderne frodi digitali, restituendo all'utente il pieno controllo e la sicurezza della propria identità digitale.

Ringraziamenti

Bibliografia

- **[1]** Domingos, P. *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World*.
- **[2]** Tegmark, M. *Life 3.0: essere umani nell'era dell'intelligenza artificiale*.
- **[5]** Hawking, S. *Brief Answers to the Big Questions*.
- **[6]** Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*.
- **[7]** Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow.*
- **[8]** Bishop, C. M. *Pattern Recognition and Machine Learning*.
- **[13]** Müller, A. C., & Guido, S. *Introduction to Machine Learning with Python*.
- **[16]** James, G., Witten, D., Hastie, T., & Tibshirani, R. *Introduction to Statistical Learning*.
- **[21]** Lininger, R., & Vines, R. D. *Phishing: Cutting the Identity Theft Line*.
- **[23]** Mitnick, K. D., & Simon, W. L. *The Art of Deception: Controlling the Human Element of Security*.

Articoli Scientifici e Pubblicazioni

- **[18]** *Dam Minh Linh, Ha Duy Hung, Han Minh Chau, Quang Sy Vu, Thanh-Nam Tran*, 2024. "Real-time phishing detection using deep learning methods by extensions".
- **[19]** *Muon Ha, Yulia Shichkina, Son Thanh Phan, Nhan Nguyen*, 2023
- "Classification of Malicious Websites Using Machine Learning Based on URL Characteristics".
- **[22]** *Rana Alabdan*, 2020."A Survey of Phishing Attacks: Their Types, Vectors, and Technical Approaches to Mitigate".
- **[24]** *Ikuomola, A. J., Ogunbameru, A. and Nwanze, M.N*, 2020. "Development of a malicious website detection system using web browser extension".
- **[25]** *M. Amir Syafiq Rohmat Rose, Nurlida Basir, Nur Fatin Nabila Rafie Heng, Nurzi Juana Mohd Zaizi, Madihah Mohd Saudi* , 2022. "Phishing Detection and Prevention using Chrome Extension".
- **[26]** *Veni S. , Mr. K. Arun Kumar*, 2024. "PhishShield: ML Based-Powered Phishing [Client-Side Protection Against Web Spoofing Attacks]".
- **[27]** *Mrs.S.Suma , S Reshma , Santhosh S , Roshini S*, 2023. "BrowSEcure".